第4回　ニューラルネットワークの最適化・正則化

講義講師：飯山 燈
演習講師：鄭 晟徹

©Yutaka Matsuo. All rights reserved.

全体の位置づけ

#	タイトル
1	人工知能の概要とディープラーニングの意義
2	機械学習基礎
3	ニューラルネットワーク基礎
4	ニューラルネットワークの最適化・正則化
5	畳み込みニューラルネットワーク
6	深層学習と画像認識
7	回帰結合型ニューラルネットワークと系列データ処理
8	Transformer基礎
9	大規模言語モデルと自然言語処理
10	表現学習と自己教師あり学習
11	深層強化学習
12	深層生成モデル

©Yutaka Matsuo. All rights reserved.

今回の目的と目標

- **目的**
  - ニューラルネットワークの基礎を踏まえ，最適化・初期化・正則化などのより実践的な内容を理解する

- **目標**
  - 確率的勾配降下法（SGD）や各種最適化アルゴリズムについて基本的な概念を説明できる
  - メジャーなパラメータ初期化方法について説明できる
  - メジャーな正規化手法について説明できる
  - 様々な過学習を抑える手法について説明できる

©Yutaka Matsuo. All rights reserved.

目次

- 最適化アルゴリズム（Optimizer）
- パラメータの初期化戦略
- 正規化
- 正則化
- 実践的な方法論
- 深層学習ライブラリ・CPU・GPU

©Yutaka Matsuo. All rights reserved.

目次

- **最適化アルゴリズム（Optimizer）**
- パラメータの初期化戦略
- 正規化
- 正則化
- 実践的な方法論
- 深層学習ライブラリ・CPU・GPU

©Yutaka Matsuo. All rights reserved.

ミニバッチ勾配降下法（おさらい）

- データ集合全て（バッチ）を使って学習するのではなく，一部のサンプル（ミニバッチ）を毎回ランダムに選択して学習する方法

  - データ集合から，サンプルを「確率的」に選択して誤差関数を求める
  - 一回のサンプルあたりに含まれるデータの数をバッチサイズという
    - バッチサイズを1としたミニバッチ勾配降下法を確率的勾配降下法（Stochastic Gradient Descent; SGD）という
  - 確率的にデータを選択することで，最適化が局所解に固定されにくくなる
  - 全データ集合を使って学習しても収束してない場合は，もう何周か回してみる
  - データ集合を1周回す単位をエポックという
    - 深層学習だと基本的に複数エポック必要
    - エポックごとにデータをシャッフルし，分割して順番に使用する

©Yutaka Matsuo. All rights reserved.

パラメータの更新手法

- （確率的）勾配降下法では勾配に学習率をかけて更新した

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta \nabla E\left(\boldsymbol{\theta}^{(t)}\right)$$

  適切なパラメータ（最小は無理でもそれに近い極小）を求めるためには
  学習率，つまり更新の速度をうまく設定する必要がある
  - 初めは大きく動いて大域的に値が小さい領域を探したい
    - 局所最適解に囚われたくない
    - 最初から小刻みだと学習に時間がかかってしまう
  - 学習終盤では細かく更新して極小値を探したい
    - 学習率が大き過ぎると収束しなくなってしまう
- 最適化にあたり，Pathological Curvature（詳細は補足を参照）などの問題があり，その克服が課題となっている

  → 更新量を決定する**最適化アルゴリズム（Optimizer）**を考える

©Yutaka Matsuo. All rights reserved.

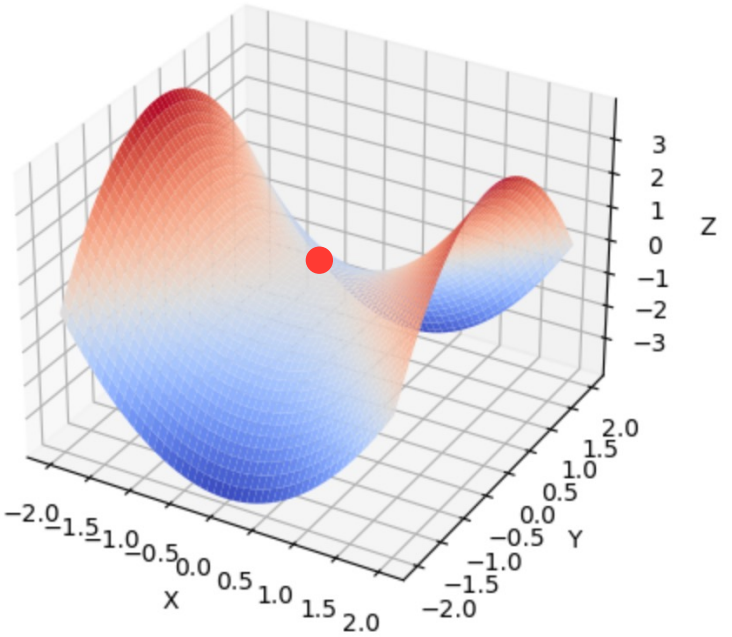
モメンタム（慣性項）

- 通常の勾配降下法では更新量 $\Delta\boldsymbol{\theta}^{(t+1)} = -\eta\nabla E\left(\boldsymbol{\theta}^{(t)}\right)$ であったが，これを

$$\Delta\boldsymbol{\theta}^{(t+1)} = -\eta\nabla E\left(\boldsymbol{\theta}^{(t)}\right) + \underbrace{\gamma\Delta\boldsymbol{\theta}^{(t)}}_{\text{モメンタム（慣性）項}}$$

  とする

- 前回の更新量$\Delta\boldsymbol{\theta}^{(t)}$を用いて，勾配の方向が変わる際に行き過ぎないようにする
- 逆に，同じ方向に進んでいる時は勢いが付く
- SGDの収束の遅さや，**鞍点（Saddle Point）**で止まる・振動してしまう等の弱点を補う
  （ただし，必ずしもSGDより良いわけではない）
  - 鞍点：ある方向から見れば極大値で，
    別方向から見れば極小値となる点



©Yutaka Matsuo. All rights reserved.

ネステロフの加速法

ネステロフの加速法（Nesterov's Accelerated Gradient Method ; NAG）

[Nesterov 1983]

- Momentum SGDを更に改良したもの
  - 勾配の評価は通常現在のパラメータに対して行うが，NAGでは更新後のパラメータを大雑把に見積もって評価に用いる

$$\Delta\boldsymbol{\theta}^{(t+1)} = -\eta\nabla E\left(\boldsymbol{\theta}^{(t)} + \gamma\Delta\boldsymbol{\theta}^{(t)}\right) + \gamma\Delta\boldsymbol{\theta}^{(t)}$$

大雑把な更新後のパラメータの推定

  - 先のパラメータを予測して動くことで収束の高速化が期待できる
  - ちなみにscikit-learnのSGDだと標準でネステロフの加速法を使っている

©Yutaka Matsuo. All rights reserved.

SGDの改善

- SGD系の手法では，勾配が急な次元（パラメータ）では速く収束するが，変化が緩やかな次元では収束が遅い

  →　高次元になると収束が遅くなってしまう

  →　各次元（各軸）ごとに学習率を調整してバランスよく更新

©Yutaka Matsuo. All rights reserved.

AdaGrad

AdaGrad（Adaptive Gradient Algorithm） [Duchi+ 2011]

- 各次元毎に学習率を調整することで問題の緩和を図る

$$\Delta\theta_i^{(t+1)} = -\frac{\eta}{\sqrt{\sum_{j=0}^{t} \nabla E\left(\theta_i^{(j)}\right)^2}} \nabla E(\theta_i^{(t)})$$

- i はパラメータの次元方向のインデックス
- 今までの勾配二乗の累積和（分母）で割り，各次元で更新量を調整している

変化が大きな次元 → 学習率 小
変化が小さな次元 → 学習率 大　となる

©Yutaka Matsuo. All rights reserved.

RMSprop

RMSprop [Tieleman & Hinton 2012]

- AdaGradは累積和で割った値を学習率としているため，学習するにつれ学習率が落ちていく性質がある
  - 学習率が一度0に近づくと再度上げることができない
  - 一度平坦になって再び勾配があるような状況では更新しづらくなる
- RMSprop
  - 重み付き移動平均を用いて直近の勾配二乗を重視する

$g_i^{(t)} = \nabla E(\theta_i^{(t)})$と表記すると

$$\Delta\theta_i^{(t+1)} = -\frac{\eta}{\sqrt{v_i^{(t)} + \epsilon}} g_i^{(t)}, \quad v_i^{(t)} = \rho v_i^{(t-1)} + (1-\rho)\left(g_i^{(t)}\right)^2 \qquad (v_0 = 0)$$

$v_i^{(t-1)}$：前回までの勾配二乗の蓄積値(重み付き移動平均)

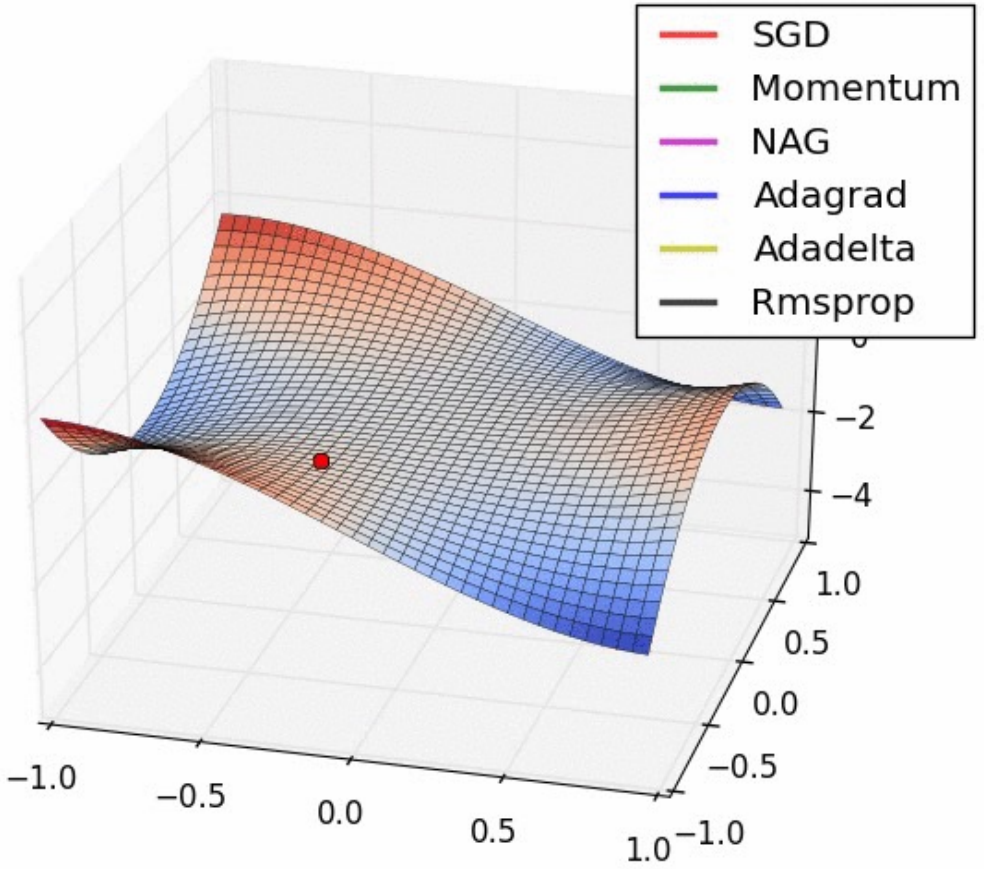
$\left(g_i^{(t)}\right)^2$：新しい勾配二乗

→ 以前の勾配二乗の影響は ρ によって減衰していく

©Yutaka Matsuo. All rights reserved.

ここまでの手法の可視化

- 各種Optimizerの挙動（gif）



https://gifer.com/en/Di0d

©Yutaka Matsuo. All rights reserved.

Adam

Adam [Kingma & Ba 2014]

- RMSPropを発展させたオプティマイザ
  - 一階・二階モメンタムを利用し，バイアス補正を導入
- 一概に”良い手法” は無いが，DL系の実装ではAdamが標準的に用いられる

$$g_i^{(t)} = \nabla E\left(\theta_i^{(t)}\right), \Delta\theta_i^{(t+1)} = -\frac{\eta}{\sqrt{\hat{v}_i{}^{(t)} + \epsilon}}\hat{m}_i{}^{(t)}$$

青字： RMSPropとの差分

- 勾配に関しても重み付き移動平均を取る：

$$m_i^{(0)} = 0, m_i^{(t)} = \beta_1 m_i^{(t-1)} + (1-\beta_1) g_i^{(t)}$$
$$v_i^{(0)} = 0, v_i^{(t)} = \beta_2 v_i^{(t-1)} + (1-\beta_2)\left(g_i^{(t)}\right)^2$$

- m, vに関しては初期値を0に設定しているため，0付近に値が偏りがち（真の値とずれてしまう）

  例： $\beta_1 = 0.9$とすると

$$m_i^{(1)} = 0.9 \cdot 0 + (1-0.9) g_i^{(1)} = 0.1 g_i^{(1)}$$
$$m_i^{(2)} = 0.9 m_i^{(1)} + 0.1 g_i^{(2)} = 0.09 g_i^{(1)} + 0.1 g_i^{(2)}$$

→バイアス補正を行う（$\hat{m}$, $\hat{v}$）

$$\hat{m}_i^{(t)} = \frac{m_i^{(t)}}{1-\beta_1^t}, \qquad \hat{v}_i{}^{(t)} = \frac{v_i^{(t)}}{1-\beta_2^t}$$

©Yutaka Matsuo. All rights reserved.

Adamの派生

Adamax [Kingma & Ba 2014]

- Adamの二階モメンタム $\boldsymbol{v}$ の$\boldsymbol{L_2}$ノルムを$\boldsymbol{L_p}$ノルムに拡張
- 過去のモメンタムより大きな勾配絶対値が来た場合に更新
  → 計算効率も良く，勾配の外れ値に強い

$$\boldsymbol{v}^{(t)} = \lim_{p\to\infty} \left(\boldsymbol{v}^{(t)}\right)^{\frac{1}{p}} = \lim_{p\to\infty} \left((1-{\beta_2}^p) \sum_{i=1}^{t} {\beta_2}^{p(t-i)} |\boldsymbol{g}^{(i)}|^p \right)^{\frac{1}{p}}$$

©Yutaka Matsuo. All rights reserved.

Adamの派生

Nadam [Dozat 2016]

- AdamにNAGを適用したオプティマイザ
- Adamの安定性とNAGの高速化の両立を目指す

$$\theta_i^{(t+1)} = \theta_i^{(t)} - \frac{\eta}{\sqrt{\hat{v}_i^{(t)} + \epsilon}} \left(\frac{\mu^{(t+1)} m^{(t)}}{1 - \prod_{i=1}^{t+1} \mu^{(i)}} + \frac{(1 - \mu^{(t)}) g_i^{(t)}}{1 - \prod_{i=1}^{t} \mu^{(i)}} \right)$$

AMSGrad [Reddi+ 2019]

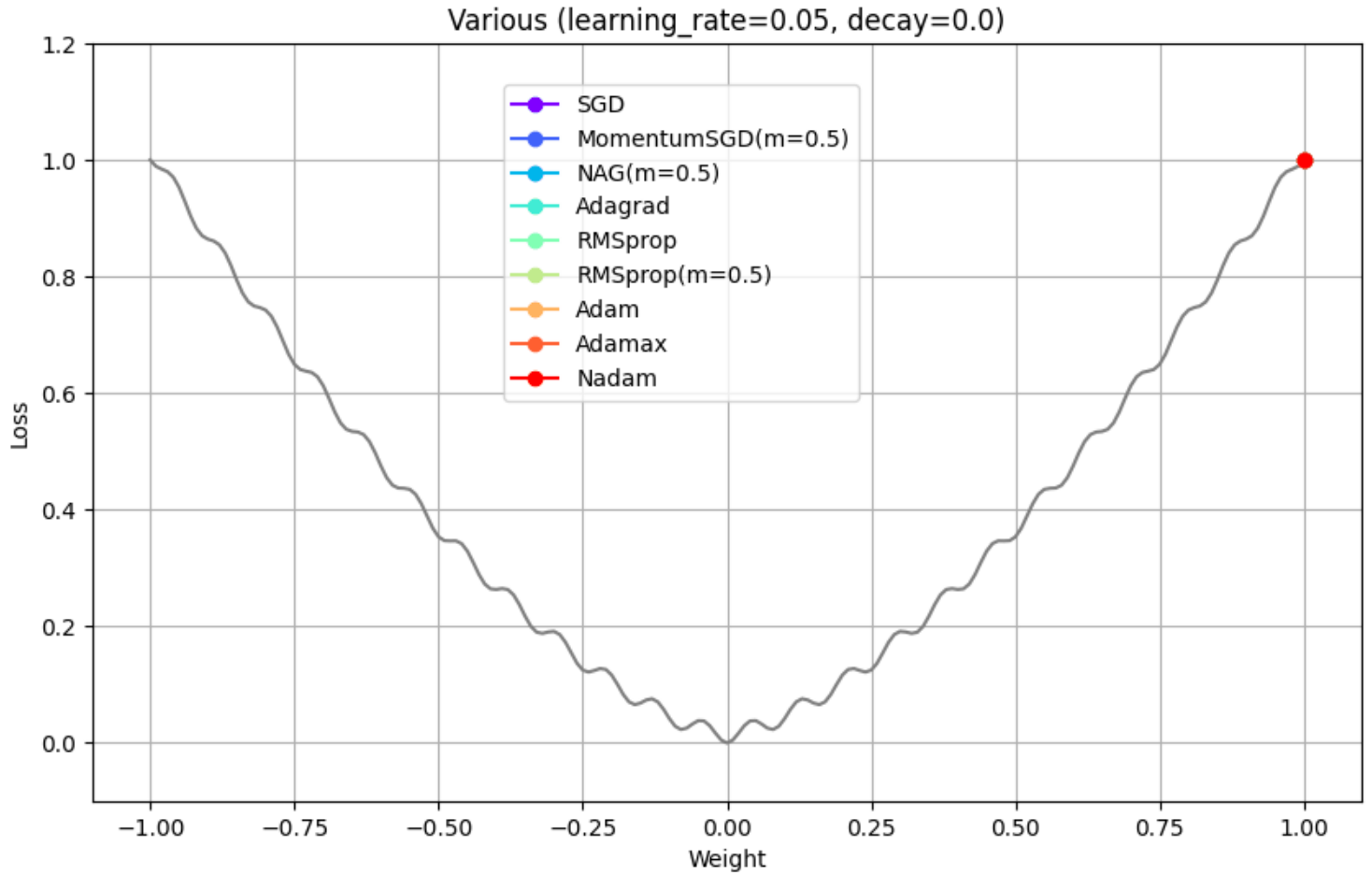
- Adamの二次モーメントの急激な変動に対処した手法
- 過去の勾配情報を使うことで解決を試みる

$$v_i^{(t)} = \beta_2 v_i^{(t-1)} + (1 - \beta_2) \left(g_i^{(t)} \right)^2$$

©Yutaka Matsuo. All rights reserved.

最適化アルゴリズムの可視化

- Momentum系の手法は行き過ぎて局所解にはまったりしている様子が分かる
- 状況によって大きく変わるので，ベストの手法というのは存在しない



©Yutaka Matsuo. All rights reserved.

目次

- 最適化アルゴリズム（Optimizer）
- **パラメータの初期化戦略**
- 正規化
- 正則化
- 実践的な方法論
- 深層学習ライブラリ・CPU・GPU

©Yutaka Matsuo. All rights reserved.

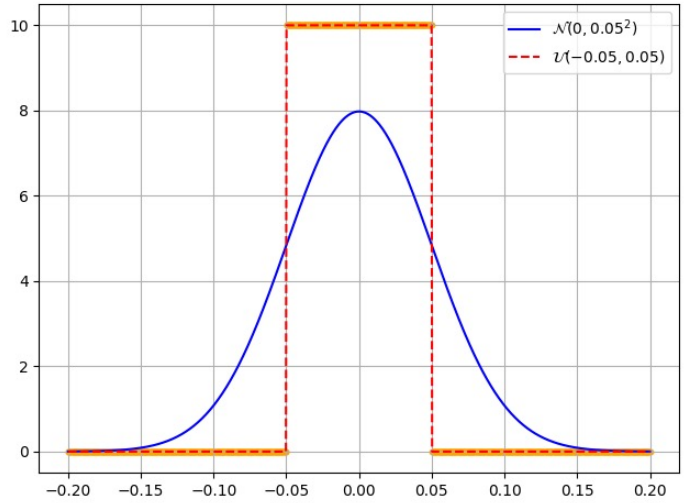
パラメータの初期化

- 例１：一定幅の一様分布（Uniform）

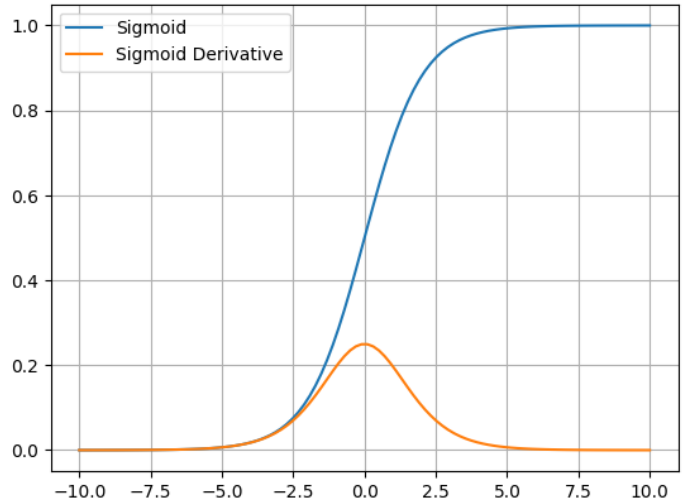
$$w_{i,j}^{l} \sim Uniform(-0.05, 0.05)$$

- 例２：平均0，標準偏差0.05のガウス分布

$$w_{i,j}^{l} \sim \mathcal{N}(0, 0.05^2)$$



- ランダム初期化の問題：勾配消失
  - うまく初期値を選ばなければ，
    活性化関数の勾配の小さい領域に入ってしまう
    → 勾配が伝達されない
  - 特に深いと顕著（何回も活性化関数をかけるため）

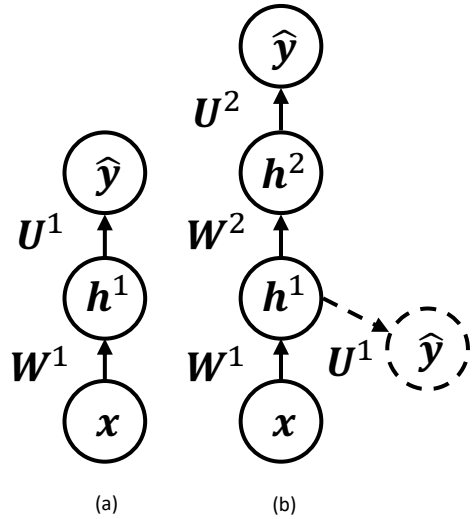


©Yutaka Matsuo. All rights reserved.

層ごとの事前学習によるパラメータ初期化

- **事前学習（Pre-Training）**：単純なタスクで単純なモデルを訓練してから，目的のタスクを訓練し実行する戦略

- **貪欲教師あり事前学習（Greedy Layer-Wise Training）**[Bengio+ 2007]
  - 教師あり学習問題をさらに単純な教師あり学習問題に分割する事前学習
  - 隠れ層が1層のニューラルネットワークを教師あり事前学習したあと(a)，隠れ層の出力を入力として受けとる層を追加して，事前学習をする(b)
  - これを必要な層の分だけ繰り返す
  - この事前学習方法によって，良好な初期パラメータが得られる



Deep Learning (Goodfellow, Bengio & Courville)

- 最近では層ごとの初期化の代わりに，特定の分布からのサンプリングで初期化する方法が取られている

©Yutaka Matsuo. All rights reserved.

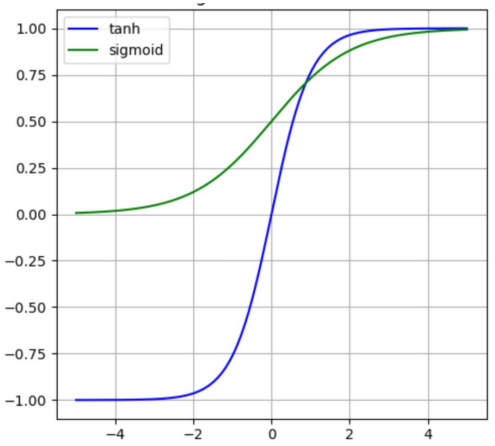
Xavier initialization

Xavierの初期化（Xavier Initialization Xavier法/Glorot法） [Glorot & Bengio 2010]

- 初期重みを層の大きさに応じた分散を持つ分布からサンプリングする

$$w_{i,j}^{l} \sim \mathcal{N}\left(0, \frac{2}{n_{in} + n_{out}}\right), \qquad w_{i,j}^{l} \sim Uniform\left(-\frac{\sqrt{6}}{\sqrt{n_{in} + n_{out}}}, \frac{\sqrt{6}}{\sqrt{n_{in} + n_{out}}}\right)$$

  - n_{in}: 入力サイズ（fan_in），n_{out}: 出力サイズ（fan_out）
    - 畳み込み層の場合，fan_inやfan_outは，チャネル数 * カーネルサイズ（幅 * 高さ）（畳み込み，チャネル，カーネルについては第5回を参照）



- Xavierの初期化の動機
  - 活性化関数がシグモイドやTanhと想定
  - このときに，うまく学習するようなパラメータ初期化にしたい
  - 入力と出力が同じ分散になるように初期化すれば良い
    - 順伝播と逆伝播で同じ分散にすることを意味する
    - シグモイドやTanhの場合は（平均0で）入力・出力の分散が1になれば活性化関数がうまく働くと考えられる

©Yutaka Matsuo. All rights reserved.

Xavier initialization

順伝播が $\boldsymbol{y} = \boldsymbol{W}^T\boldsymbol{x}$とすると，分散は $Var(\boldsymbol{y}) = n_{in} \cdot Var(\boldsymbol{W}) \cdot Var(\boldsymbol{x})$

- 入出力の分散が1になるようにすると，$Var(\boldsymbol{W}) = \frac{1}{n_{in}}$となる

さらに出力の次元数も考慮すると，分散が以下になるように初期化すればいい

$$Var(\boldsymbol{W}) = \frac{1}{\frac{n_{in} + n_{out}}{2}} = \frac{2}{n_{in} + n_{out}}$$

- すなわち，分散パラメータを$\frac{2}{n_{in}+n_{out}}$に設定したガウス分布から初期値をサンプリング

一様分布$Uniform(-a, a)$で初期化する場合は，一様分布の分散が

$$Var(\boldsymbol{W}) = \frac{(2a)^2}{12} = \frac{4a^2}{12} = \frac{a^2}{3}$$

なので， aは次のように設定する

$$\frac{a^2}{3} = \frac{2}{n_{in} + n_{out}} \Rightarrow a = \frac{\sqrt{6}}{\sqrt{n_{in} + n_{out}}}$$

©Yutaka Matsuo. All rights reserved.

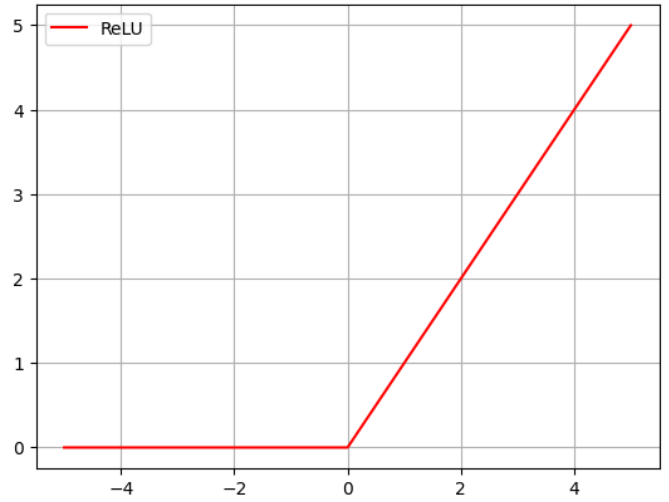
He initialization

- 活性化関数の値によって，どの位置が望ましいかは異なる
  - Xavierの初期化はシグモイドやTanhの場合

Heの初期化（He Initialization/Kaiming法/He法） [He+ 2015]

- 活性化関数としてReLUを使うことを仮定
  - ReLUの非対称性（0以下は0になる）から，分散を2倍にして

$$Var(\boldsymbol{W}) = \frac{2}{n_{in}}$$



- Heの初期化

$$w_{i,j}^{l} \sim \mathcal{N}\left(0, \frac{2}{n_{in}}\right)$$

$$w_{i,j}^{l} \sim Uniform\left(-\frac{\sqrt{6}}{\sqrt{n_{in}}}, \frac{\sqrt{6}}{\sqrt{n_{in}}}\right)$$

©Yutaka Matsuo. All rights reserved.

初期化まとめ

- 初期化の方法によって，勾配が爆発したり消失したりする
  - 活性化関数の非線形な部分に入ることが要因
  - 特に深くなると影響が顕著（Multiplicative Effect）

- 事前学習 [Bengio+ 2007] は層ごとの事前学習というアイディアで
  この問題を解消

- 近年は，初期化方法を工夫するのが普通
  - Xavierの初期化: tanhやシグモイドを活性化関数にする場合
  - Heの初期化: ReLUを活性化関数にする場合

©Yutaka Matsuo. All rights reserved.

目次

- 最適化アルゴリズム（Optimizer）
- パラメータの初期化戦略
- **正規化**
- 正則化
- 実践的な方法論
- 深層学習ライブラリ・CPU・GPU

©Yutaka Matsuo. All rights reserved.

正規化の背景

共変量シフト（Covariate Shift）

- 訓練時と推論時でデータの分布が変化すること
- 例：通常のMNISTで訓練，15度回転したMNISTでテスト
- 一般に精度が大きく低下する

内部共変量シフト（Internal Covariate Shift）

- MLPの中間表現でも共変量シフトと同様のことが起こる
- 毎度全く違う分布だと学習しにくい

→ 平均0，分散1の分布に毎回変換すれば良い（正規化）

©Yutaka Matsuo. All rights reserved.

正規化とは

- 値のばらつきを抑えるために、各値から平均を引いて分散で割ることで、値の集合の平均を 0、分散を 1 とする操作

$\boldsymbol{x} = (x_1, x_2, \cdots, x_n)$のとき

平均 $\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$, 分散 $\sigma^2 = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2$ を用いて

全ての x_i に対して $x_i = \frac{(x_i - \bar{x})}{\sqrt{\sigma^2}}$ とする

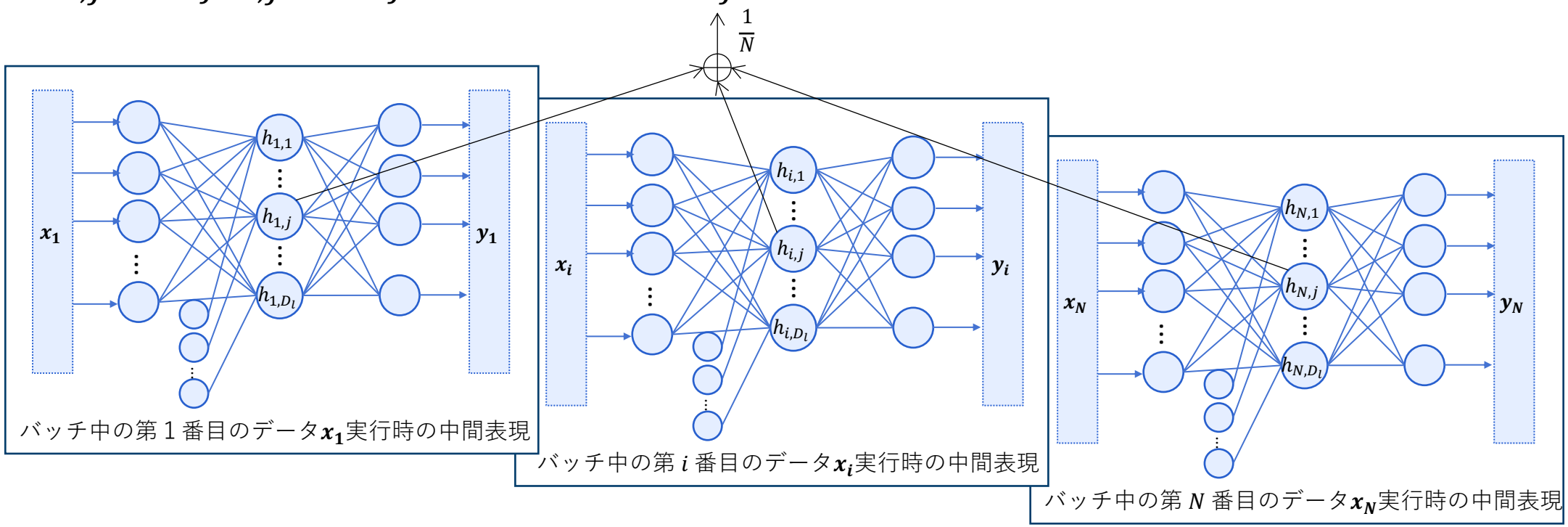
©Yutaka Matsuo. All rights reserved.

バッチ正規化 [2]

バッチ正規化（Batch Normalization） [Ioffe & Szegedy 2015]

$$h'^{BN}_{i,j} = \frac{h_{i,j} - \bar{h}_j}{\sqrt{\sigma_j^2 + \epsilon}} \qquad \bar{h}_j = \frac{1}{N}\sum_{i=1}^{N} h_{i,j}\,,\ \sigma_j^2 = \frac{1}{N}\sum_{i=1}^{N}\left(h_{i,j} - \bar{h}_j\right)^2$$

$$h^{BN}_{i,j} = \gamma_j h'^{BN}_{i,j} + \beta_j$$



©Yutaka Matsuo. All rights reserved.

Layer正規化

Layer正規化（Layer Normalization） [Ba+ 2016]

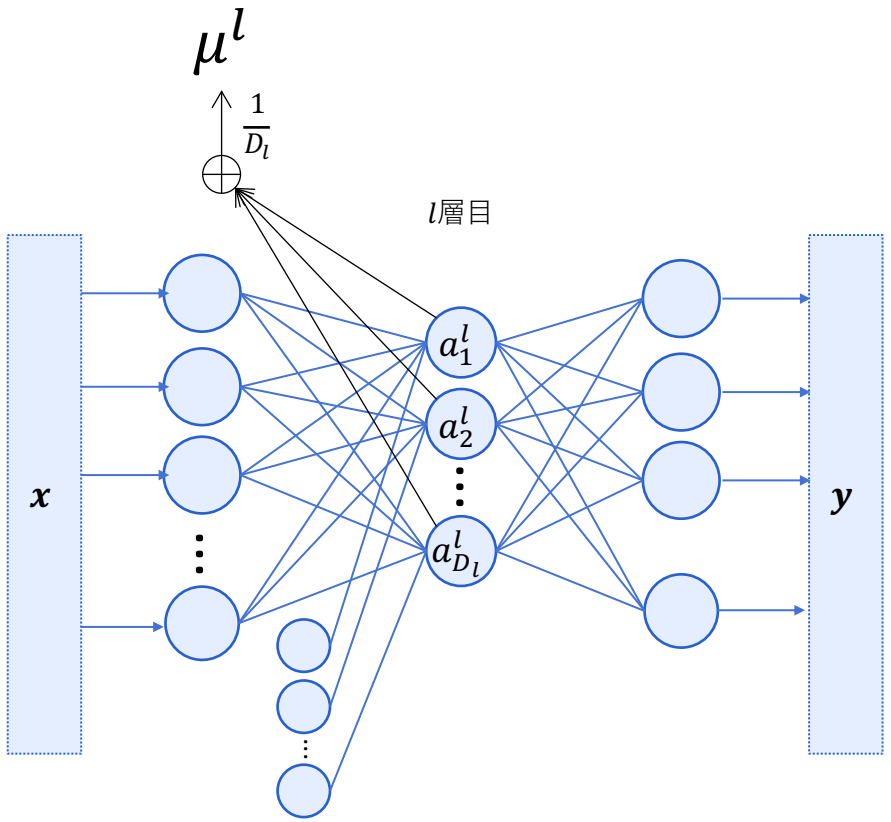
- バッチ正規化の問題点
  - データの長さ（次元）がサンプルごとに違うと適用が困難
  - バッチサイズが小さいと不安定になる

- Layer正規化
  - バッチ内の各サンプル毎に、
    各レイヤ内の全ユニットで正規化する

$$\mu^l = \frac{1}{D_l}\sum_{i=1}^{D_l} a_i^l$$

$$\sigma^l = \sqrt{\frac{1}{D_l}\sum_{i=1}^{D_l}(a_i^l - \mu^l)^2}$$

  - D_l は層 l の出力の次元

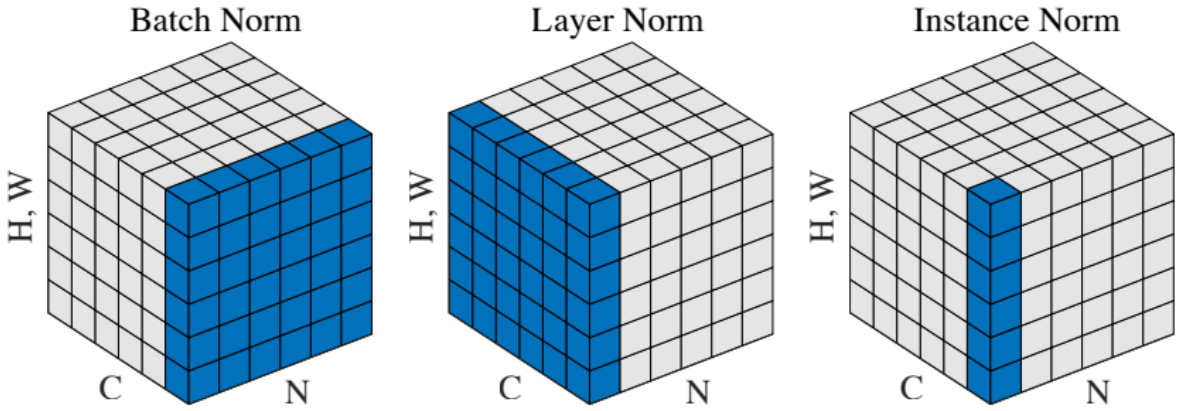


©Yutaka Matsuo. All rights reserved.

Instance正規化

Instance正規化（Instance Normalization） [Ulyanov+ 2016]

- 各サンプル・各チャネル毎に平均・分散を計算する
  - Layer正規化では、各サンプル毎に、正規化対象レイヤ内の全ユニットを全チャンネル分まとめて正規化する
- バッチサイズが小さくても安定
- 画像の画風変換の文脈で提案
  - コントラスト正規化とも呼ばれる



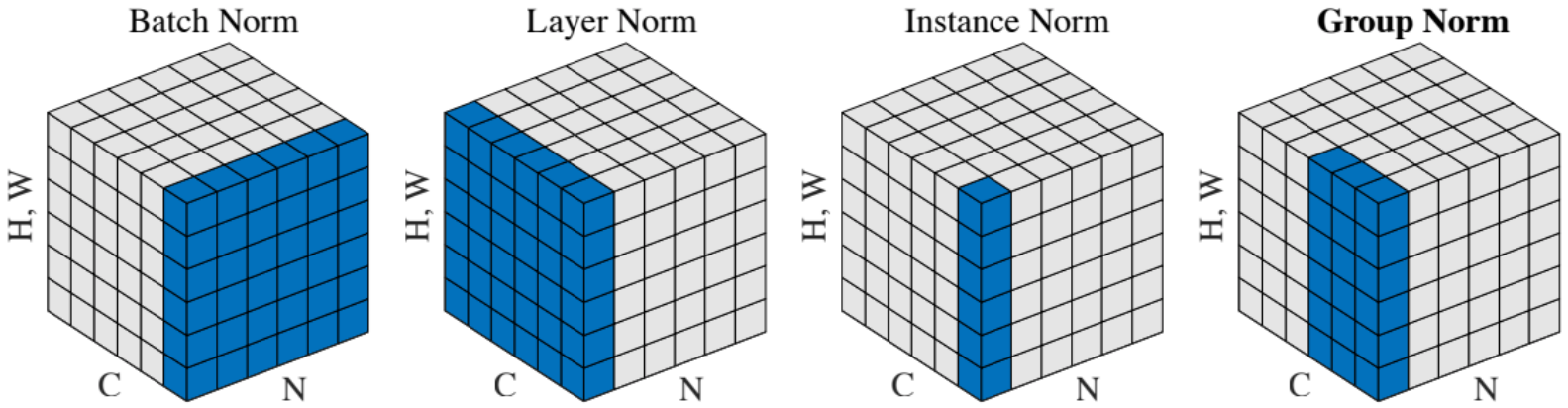
[Wu & He 2018] “Group Normalization”

©Yutaka Matsuo. All rights reserved.

Group正規化

Group正規化（Group Normalization） [Wu & He 2018]

- チャネルをいくつかのグループに分けて，そのグループの中で正規化
  - Layer正規化とInstance正規化の中間とみなせる
    - Layer正規化 → 各チャネルの特徴を考慮できない
    - Instance正規化 → チャネル間の依存関係を考慮できない
  - 平均と分散は，訓練時と推論時ともに入力データのものを使用



©Yutaka Matsuo. All rights reserved.

目次

- 最適化アルゴリズム（Optimizer）
- パラメータの初期化戦略
- 正規化
- **正則化**
- 実践的な方法論
- 深層学習ライブラリ・CPU・GPU

©Yutaka Matsuo. All rights reserved.

正則化

過学習の原因：

1. パラメータの数が多い
   - 表現力が上がるため，訓練データにより適合し過ぎやすい
2. パラメータのノルムが大きい
   - データの少しの変化に機敏に反応しやすい

→ 2の対策として，**正則化（Regularization）**を行う

©Yutaka Matsuo. All rights reserved.

L2正則化

- モデルのパラメータwの**L2正則化（L2 Regularization）**は以下の式で定義される

$$\Omega(\boldsymbol{\theta}) = \frac{1}{2}\|\boldsymbol{w}\|_2^2$$

- 正則化された目的関数とその勾配

$$\tilde{E}(\boldsymbol{\theta}) = E(\boldsymbol{\theta}) + \frac{\alpha}{2}\|\boldsymbol{w}\|_2^2$$
$$\nabla_w \tilde{E}(\boldsymbol{\theta}) = \nabla_w E(\boldsymbol{\theta}) + \alpha \boldsymbol{w}$$

- この正則化手法は一般的には**重み減衰（Weight Decay）**として知られている（α は係数）
- 重み減衰項の追加によって学習則が変更され，通常の勾配の更新に一定の割合で重みベクトルが加わる
  - 重みが大きいほど大きく更新され，小さいほど更新は減る

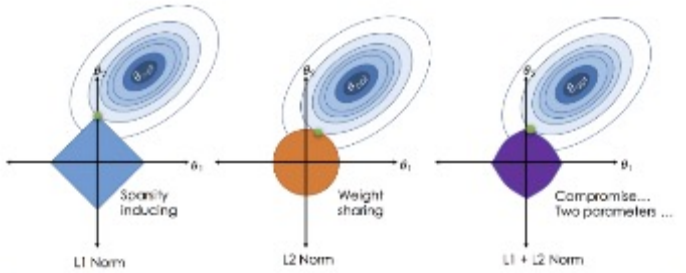
©Yutaka Matsuo. All rights reserved.

L1正則化

- モデルのパラメータwの**L1正則化（L1 Regularization）**は以下の式で定義される

$$\Omega(\boldsymbol{\theta}) = \|\boldsymbol{w}\|_1 = \sum_i |w_i|$$

  - ここで，$\boldsymbol{w} = (w_1, \ldots, w_i, \ldots, w_I)$ としている
  - $\|\boldsymbol{w}\|_1$はL1ノルム（個々のパラメータ w_i の絶対値の総和）



https://towardsdatascience.com/from-linear-regression-to-ridge-regression-the-lasso-and-the-elastic-net-4eaecaf5f7e6

- 正則化された目的関数とその勾配

$$\tilde{E}(\boldsymbol{\theta}) = E(\boldsymbol{\theta}) + \alpha\|\boldsymbol{w}\|_1$$

$$\nabla_w \tilde{E}(\boldsymbol{\theta}) = \nabla_w E(\boldsymbol{\theta}) + \alpha \mathrm{sign}(\boldsymbol{w})$$

- L2正則化と比較して，L1正則化ではよりスパースな解が得られる
  - すなわちパラメータの最適値の多くが0となる（もしくは0に近づく）
- L1正則化におけるスパース性は特徴量選択のメカニズムとして広く用いられている

©Yutaka Matsuo. All rights reserved.

スパース表現

- **スパース表現（Sparse Representation）**
  L1ペナルティによってモデルパラメータのスパース性が得られたように，
  表現のスパース性について考えることができる

$$\underset{\boldsymbol{x} \in \mathbb{R}^m}{\begin{bmatrix} -14 \\ 1 \\ 19 \\ 2 \\ 23 \end{bmatrix}} = \underset{\boldsymbol{B} \in \mathbb{R}^{m \times n}}{\begin{bmatrix} 3 & -1 & 2 & -5 & 4 & 1 \\ 4 & 2 & -3 & -1 & 1 & 3 \\ -1 & 5 & 4 & 2 & -3 & -2 \\ 3 & 1 & 2 & -3 & 0 & -3 \\ -5 & 4 & -2 & 2 & -5 & -1 \end{bmatrix}} \underset{\boldsymbol{h} \in \mathbb{R}^n}{\begin{bmatrix} 0 \\ 2 \\ 0 \\ 0 \\ -3 \\ 0 \end{bmatrix}}$$

- 上例において $\boldsymbol{h}$ はデータ $\boldsymbol{x}$ の関数であり，$\boldsymbol{B}$ を基底行列として，$\boldsymbol{x}$ の情報をスパースベクトル $\boldsymbol{h}$ で表現する

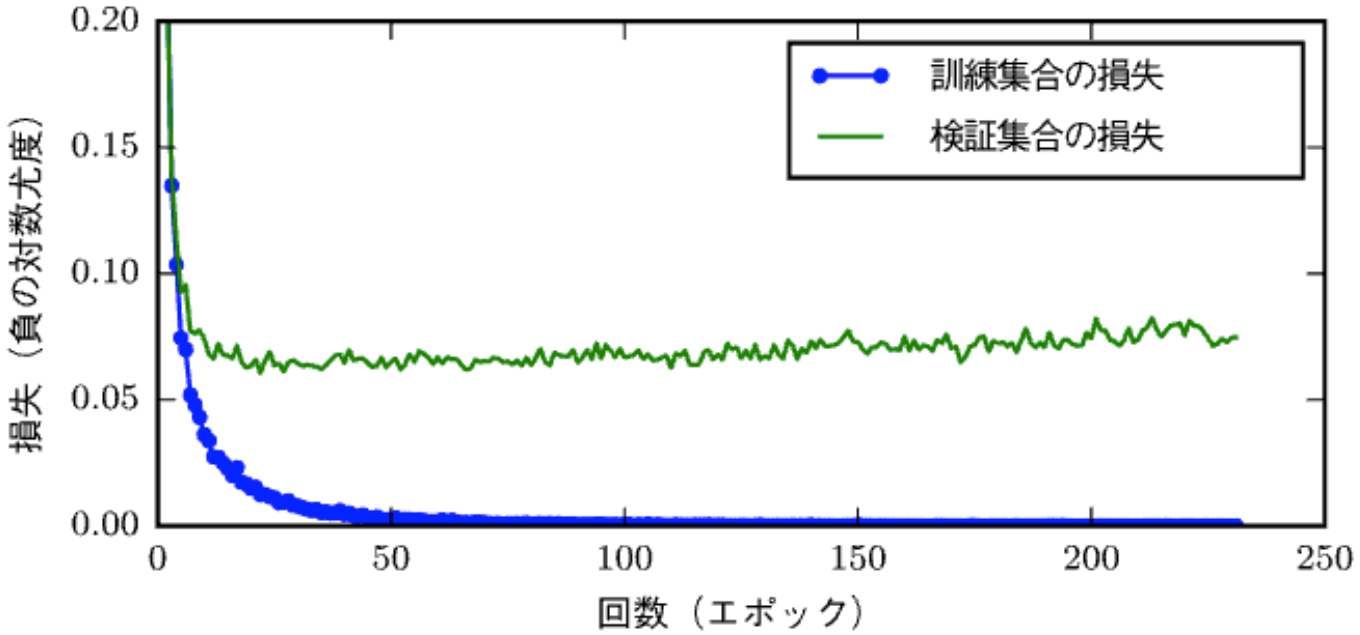
- パラメータのL1ペナルティと同じく，表現のL1ペナルティ $\Omega(\boldsymbol{h}) = \|\boldsymbol{h}\|_1 = \sum_i |h_i|$ によって表現のスパース性を得ることができる．正則化された損失関数は

$$\tilde{E}(\boldsymbol{\theta}) = \alpha \|\boldsymbol{h}\|_1 + E(\boldsymbol{\theta})$$

©Yutaka Matsuo. All rights reserved.

早期終了

- 過学習は，訓練誤差は減少するが，検証誤差が増加してしまう状態



- 適度な学習回数で訓練を終了させれば，過学習を抑えることができる

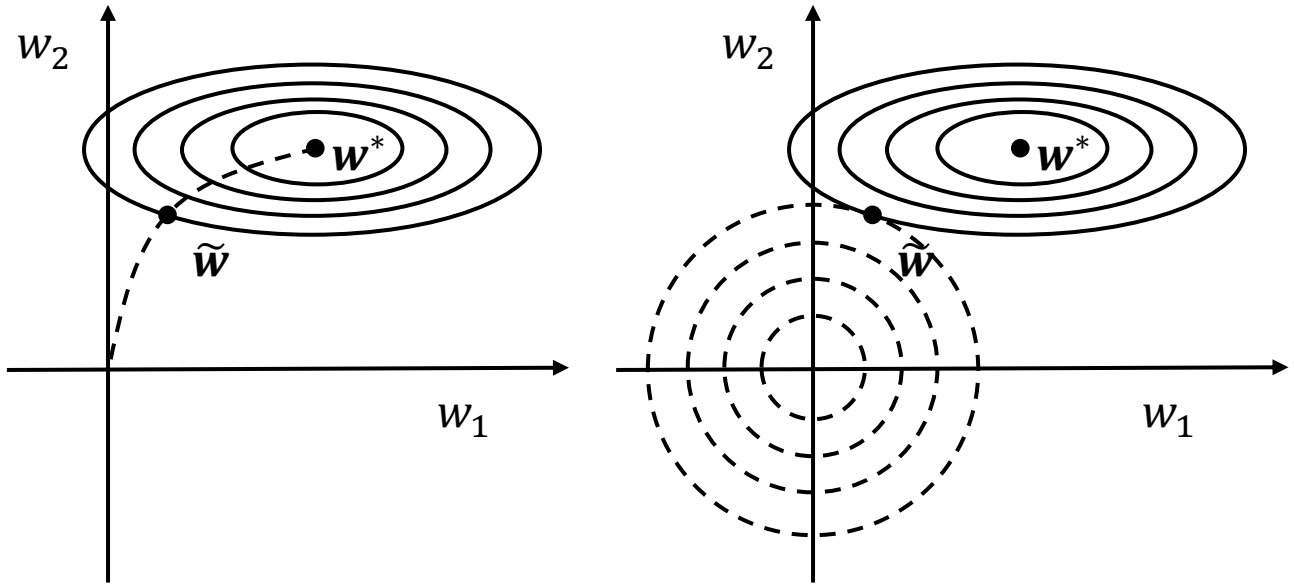
→ **早期終了（Early Stopping）**

©Yutaka Matsuo. All rights reserved.

早期終了

- 早期終了による訓練の終了回数の決定方法
  - 訓練中に定期的に（訓練集合から分割した）検証集合における検証誤差を評価する
  - 検証誤差が任意の数（ハイパーパラメータ）だけ連続で悪化したら訓練を終了し，最も検証誤差の小さいモデルを選ぶ

- 早期終了は正則化手法の一つと見做せる
  - 早期終了は訓練誤差を最小化する点 $\boldsymbol{w}^*$ の手前 $\tilde{\boldsymbol{w}}$ で終了する（図左）ので，L2パラメータ正則化（図右）などと同様に正則化として振る舞う



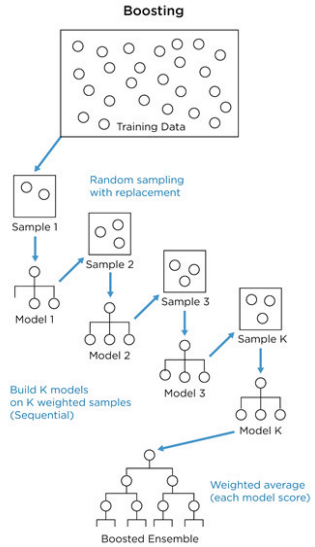
©Yutaka Matsuo. All rights reserved.

アンサンブル学習

- **アンサンブル（Ensemble）**
  複数の学習器を組み合わせて訓練させることで汎化性能を向上させる技術
  - 個々の学習器はベース学習器（Base Learner）や弱学習器（Weak Learner）と呼ばれる
  - 最終的には，分類問題であれば多数決，回帰であれば（重み付き）平均をとる
  - 代表的な手法に**ブースティング（Boosting）**，**バギング（Bagging）**，**スタッキング（Stacking）**がある

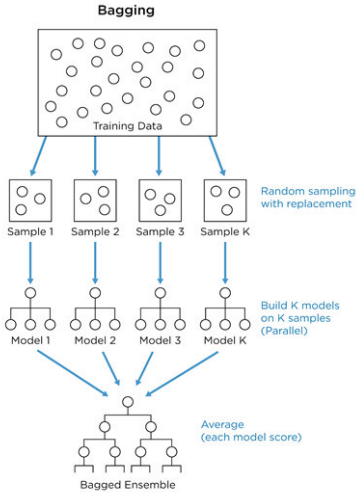
ブースティング

誤分類したデータのサンプル確率を上げながら，ベース学習器を連続的に学習させていく
例：AdaBoostなど



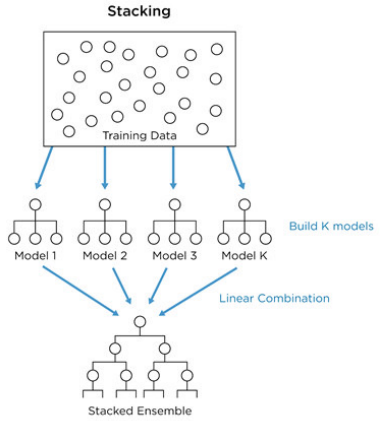
バギング

ベース学習器をデータセットの異なる部分集合に対して独立に学習させる
例：Random forest



スタッキング

異なるアルゴリズムのベース学習器をデータセット全体に対して独立に学習させ，その結果を元にメタ学習器とよばれる上位の学習器でさらに学習をおこなう



Z.-H. Zhou, “Ensemble Learning”, *Encyclopedia of Biometrics* (2009)

J. Budzik, “Many Heads Are Better Than One: The Case For Ensemble Learning”, https://www.kdnuggets.com/2019/09/ensemble-learning.html, (2019/11/21閲覧)

©Yutaka Matsuo. All rights reserved.

AdaBoost

- 弱学習器を直列につなげる際，直前の弱学習器で誤ったサンプルに対する重みを大きくして学習

- アルゴリズム
  - ラベルの取りうる値は-1または1とする
  - $m\ (m = 1, \ldots, M)$ 番目の弱学習器の，学習データ $i\ (i = 1, \ldots, N)$に関する重みを w_i^m で表す（Mは弱学習器の個数，Nは学習データ数）

1.最初の弱学習器の学習データに関する重みを初期化：$w_i^1 = \frac{1}{N}$

2.以下3-5をm番目の弱学習器について繰り返す

3.弱学習器を以下の誤差関数を最小化するように学習：$E^m = \frac{\sum_i w_i^m \left[\hat{y}^m(x_i) \neq y_i\right])}{\sum_i w_i^m}$

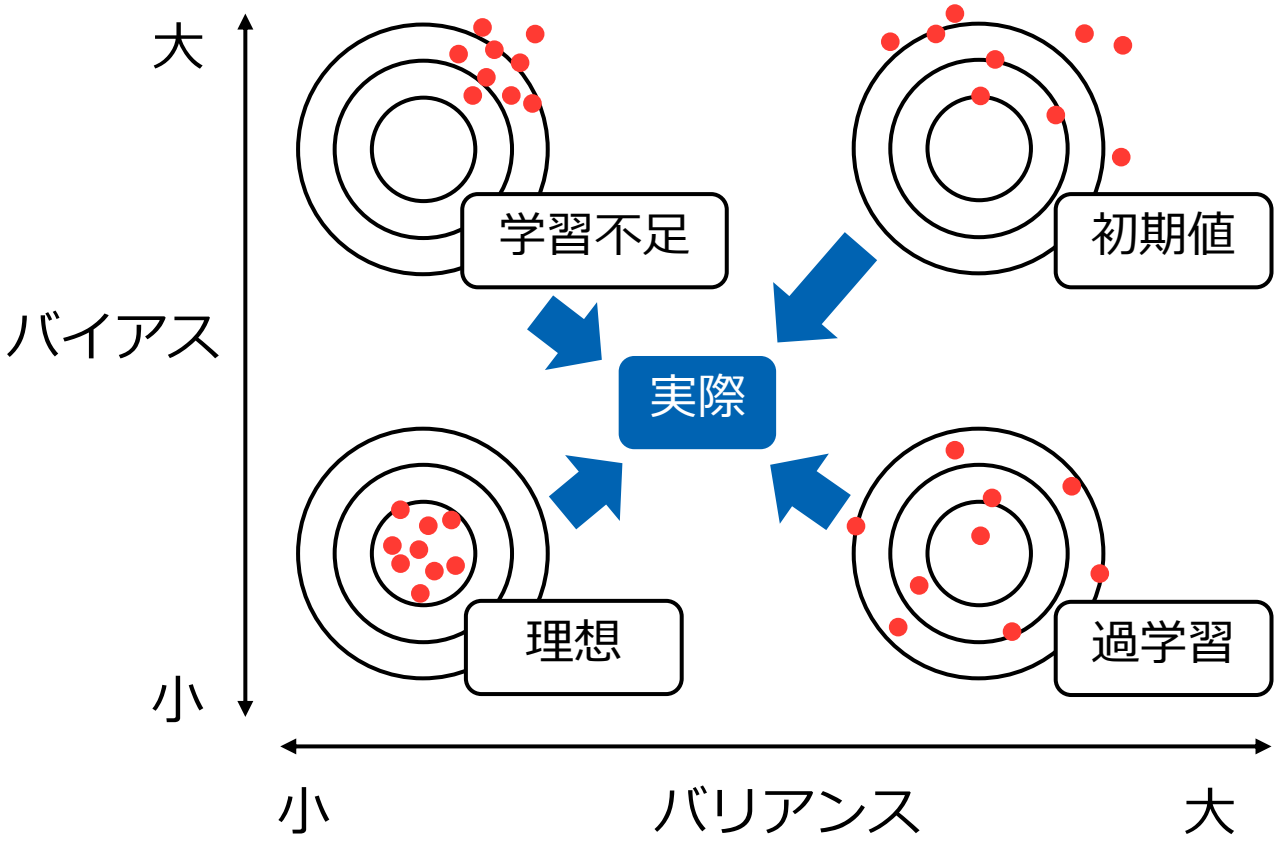
4.学習した弱学習器に対する重みを以下の式で決定：$\alpha^m = \ln(\frac{1-E^m}{E^m})$

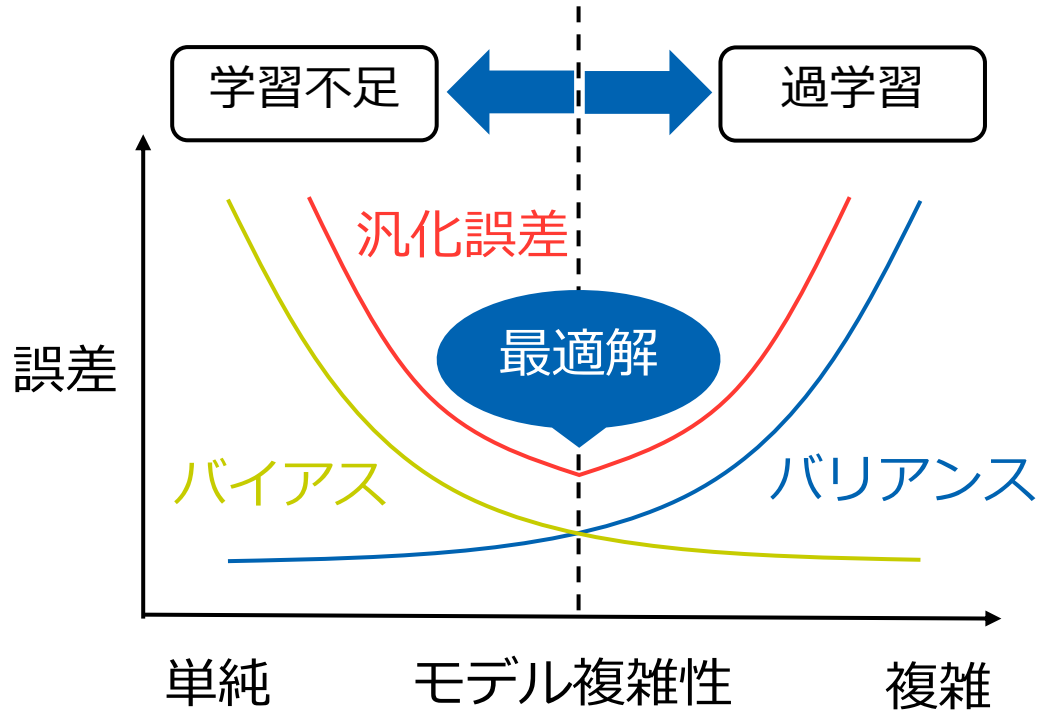
5.学習データに対する重みを以下の式で更新：$w_i^{m+1} = w_i^m \exp(-\alpha_m \hat{y}^m(x_i) y_i)$

©Yutaka Matsuo. All rights reserved.

バイアス・バリアンスと汎化誤差

- **バイアス（Bias）**：予測値と真の値の差
- **バリアンス（Variance）**：値のばらつき（分散）





©Yutaka Matsuo. All rights reserved.

期待値と分散の公式補足

バイアス・バリアンス分解では以下の式を用います。

$$\mathbb{E}(X+Y) = \mathbb{E}(X) + \mathbb{E}(Y)$$
$$\mathbb{E}(aX) = a\mathbb{E}(X), \mathbb{E}(a) = a$$
$$\mathbb{E}\{(X+Y)^2\} = \mathbb{E}(X^2) + \mathbb{E}(Y^2) + 2\mathbb{E}(X \cdot Y)$$
$$\mathbb{E}\Big(X - \mathbb{E}(X)\Big) = \mathbb{E}(X) - \mathbb{E}(X) = 0$$

アンサンブル学習のバリアンス計算では以下の式を用います。

$$\mathrm{Var}(X+Y) = \mathrm{Var}(X) + \mathrm{Var}(Y) + 2 \cdot \mathrm{Cov}(X,Y)$$
$$\mathrm{Var}(aX) = a^2\mathrm{Var}(X)$$

共分散

©Yutaka Matsuo. All rights reserved.

バイアス・バリアンス分解

- バイアス・バリアンス分解：予測誤差=バリアンス+(バイアス)^2+ノイズ

※ $\hat{f}_{\mathcal{D}}(\boldsymbol{x})$：データ集合 $\mathcal{D}$ を学習して得た学習器
$\mathbb{E}_{\mathcal{D}}[\cdot]$：$\mathcal{D}$ に関する期待値

$$\mathbb{E}_{\mathcal{D}}\left[\left(\hat{f}_{\mathcal{D}}(\boldsymbol{x})-f(\boldsymbol{x})\right)^2\right]$$
$$=\mathbb{E}_{\mathcal{D}}\left[(\hat{f}_{\mathcal{D}}(\boldsymbol{x})-\mathbb{E}_{\mathcal{D}}\left[\hat{f}_{\mathcal{D}}(\boldsymbol{x})\right]+\mathbb{E}_{\mathcal{D}}\left[\hat{f}_{\mathcal{D}}(\boldsymbol{x})\right]-f(\boldsymbol{x}))^2\right]$$
$$=\mathbb{E}_{\mathcal{D}}\left[((\hat{f}_{\mathcal{D}}(\boldsymbol{x})-\mathbb{E}_{\mathcal{D}}\left[\hat{f}_{\mathcal{D}}(\boldsymbol{x})\right])+(\mathbb{E}_{\mathcal{D}}\left[\hat{f}_{\mathcal{D}}(\boldsymbol{x})\right]-f(\boldsymbol{x})))^2\right]$$
$$=\mathbb{E}_{\mathcal{D}}\left[(\hat{f}_{\mathcal{D}}(\boldsymbol{x})-\mathbb{E}_{\mathcal{D}}\left[\hat{f}_{\mathcal{D}}(\boldsymbol{x})\right])^2\right]+\mathbb{E}_{\mathcal{D}}\left[(\mathbb{E}_{\mathcal{D}}\left[\hat{f}_{\mathcal{D}}(\boldsymbol{x})\right]-f(\boldsymbol{x}))^2\right]$$

定数なので外側の$\mathbb{E}_{\mathcal{D}}$は消してよい

$$+2\mathbb{E}_{\mathcal{D}}\left[(\hat{f}_{\mathcal{D}}(\boldsymbol{x})-\mathbb{E}_{\mathcal{D}}\left[\hat{f}_{\mathcal{D}}(\boldsymbol{x})\right])(\mathbb{E}_{\mathcal{D}}\left[\hat{f}_{\mathcal{D}}(\boldsymbol{x})\right]-f(\boldsymbol{x}))\right]$$

定数なので$\mathbb{E}_{\mathcal{D}}$の外に出せる

$$=\mathbb{E}_{\mathcal{D}}\left[(\hat{f}_{\mathcal{D}}(\boldsymbol{x})-\mathbb{E}_{\mathcal{D}}\left[\hat{f}_{\mathcal{D}}(\boldsymbol{x})\right])^2\right]+(\mathbb{E}_{\mathcal{D}}\left[\hat{f}_{\mathcal{D}}(\boldsymbol{x})\right]-f(\boldsymbol{x}))^2$$
$$+2\left(\mathbb{E}_{\mathcal{D}}\left[\hat{f}_{\mathcal{D}}(\boldsymbol{x})\right]-f(\boldsymbol{x})\right)\mathbb{E}_{\mathcal{D}}\left[\hat{f}_{\mathcal{D}}(\boldsymbol{x})-\mathbb{E}_{\mathcal{D}}\left[\hat{f}_{\mathcal{D}}(\boldsymbol{x})\right]\right]$$

$=0$

$$=\mathbb{E}_{\mathcal{D}}\left[(\hat{f}_{\mathcal{D}}(\boldsymbol{x})-\mathbb{E}_{\mathcal{D}}\left[\hat{f}_{\mathcal{D}}(\boldsymbol{x})\right])^2\right]+(\mathbb{E}_{\mathcal{D}}\left[\hat{f}_{\mathcal{D}}(\boldsymbol{x})\right]-f(\boldsymbol{x}))^2$$
$$=\mathrm{Var}\left[\hat{f}_{\mathcal{D}}(\boldsymbol{x})\right]+(\mathrm{Bias}\left[\hat{f}_{\mathcal{D}}(\boldsymbol{x})\right])^2$$

©Yutaka Matsuo. All rights reserved.

アンサンブル学習とバイアス・バリアンス

- アンサンブル学習で汎化性能が向上する理由
  - K個の弱学習器 $\hat{f}_{\mathcal{D}}^{1}(\boldsymbol{x}), \ldots, \hat{f}_{\mathcal{D}}^{K}(\boldsymbol{x})$ の平均をとってアンサンブルする場合：$\hat{f}_{\mathcal{D}}(\boldsymbol{x}^*) = \sum_k \hat{f}_{\mathcal{D}}^{k}(\boldsymbol{x})$

$$
\begin{aligned}
&\mathrm{Bias}\left[\hat{f}_{\mathcal{D}}(\boldsymbol{x})\right] \\
&= f(\boldsymbol{x}) - \mathbb{E}_{\mathcal{D}}\left[\hat{f}_{\mathcal{D}}(\boldsymbol{x})\right] \\
&= f(\boldsymbol{x}) - \frac{1}{K}\sum_k \hat{f}_{\mathcal{D}}^{k}(\boldsymbol{x}) \\
&= \frac{1}{K}\sum_k \left(f(\boldsymbol{x}) - \hat{f}_{\mathcal{D}}^{k}(\boldsymbol{x})\right) \\
&= \frac{1}{K}\sum_k \mathrm{Bias}\left[\hat{f}_{\mathcal{D}}^{k}(\boldsymbol{x})\right] \\
&= \frac{1}{K}\sum_k \delta \\
&= \delta
\end{aligned}
$$

弱学習器のバイアスが一定と仮定

$$
\begin{aligned}
&\mathrm{Var}\left[\hat{f}_{\mathcal{D}}(\boldsymbol{x})\right] \\
&= \mathrm{Var}\left[\frac{1}{K}\sum_k \hat{f}_{\mathcal{D}}^{k}(\boldsymbol{x})\right] \\
&= \left(\frac{1}{K}\right)^2 \left(\sum_k \mathrm{Var}\left[\hat{f}_{\mathcal{D}}^{k}(\boldsymbol{x})\right] + \sum_{k\neq k'} \mathrm{Cov}[\hat{f}_{\mathcal{D}}^{k}(\boldsymbol{x}), \hat{f}_{\mathcal{D}}^{k'}(\boldsymbol{x})]\right) \\
&= \left(\frac{1}{K}\right)^2 \left(\sum_k \sigma_*^2 + \sum_{k\neq k'} \sigma_{**}\right) \\
&= \left(\frac{1}{K}\right)^2 \left(K\sigma_*^2 + K(K-1)\sigma_{**}\right) \\
&= \left(\frac{1}{K}\right)\sigma_*^2 + \left(\frac{K-1}{K}\right)\sigma_{**}
\end{aligned}
$$

共分散

全ての弱学習器・弱学習器間で一定と仮定

弱学習器が独立ならば0

Kを大きくすると0に近づく

→ バイアスには影響しないが，バリアンスが小さくなるため汎化性能向上

©Yutaka Matsuo. All rights reserved.

アンサンブル学習とバイアス・バリアンス

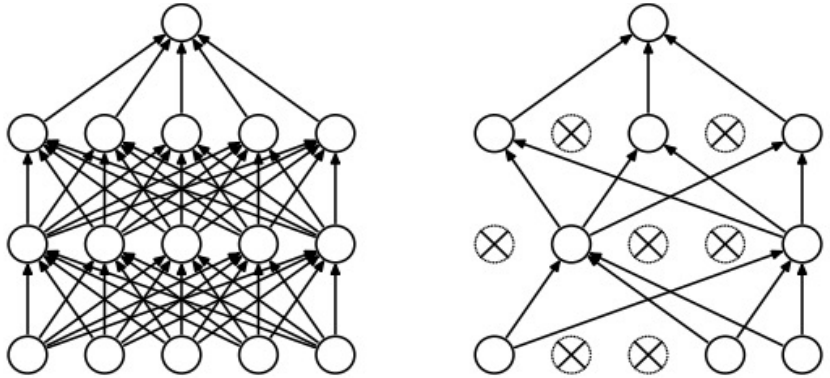
- 各手法とバイアス・バリアンスへの影響
  - バギング：多数のモデルを用いるためバリアンスを低下させる
  - ブースティング：誤りを修正していくためバイアスを低下させる
  - スタッキング：バイアスとバリアンスをバランスよく調整する

©Yutaka Matsuo. All rights reserved.

ドロップアウト

ドロップアウト（Dropout） [Srivastava+ 2014]

- 層のユニットの出力を一定の割合でマスクする（0に設定する）



Srivastava et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting” (2014)

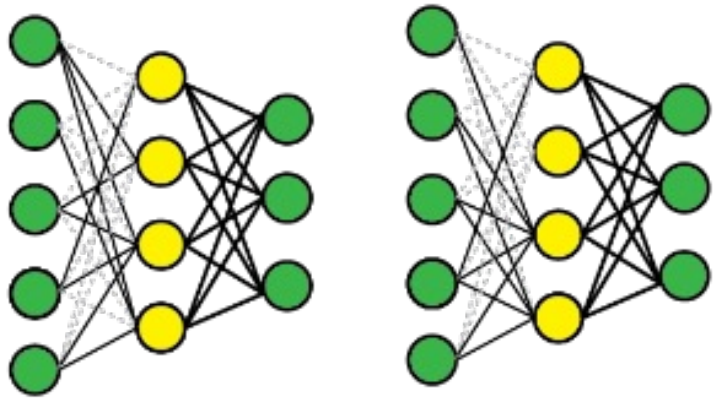
- 学習時
  - 順伝播：バッチごとにユニットを確率 p で選ぶ（実装上は0/1のマスクをランダムにサンプルしてかける）．$1-p$ がドロップアウトの割合（消去されるユニットの割合）
  - 逆伝播：選ばれたサブネットワーク上で逆伝搬
- 推論時：全部のユニットを使うが，出力スケールを合わせる（p 倍）
  - 消去されなかった割合をかけることに注意する

©Yutaka Matsuo. All rights reserved.

ドロップコネクト

ドロップコネクト（DropConnect）[Wan+ 2013]

- ドロップアウトと異なり，ノードでなく重みを一定の割合でマスクする（0に設定する）
- 推論は全結合層の出力（活性化関数の前）のサンプリングに基づいて行う
  - 図の点線の部分がマスクされた重み

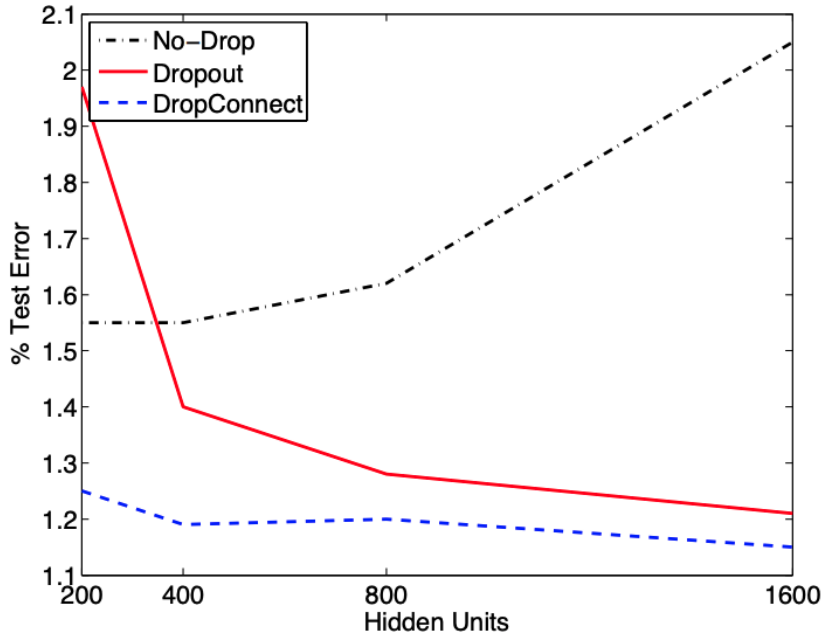


https://stats.stackexchange.com/questions/201569/what-is-the-difference-between-dropout-and-drop-connect より図を引用

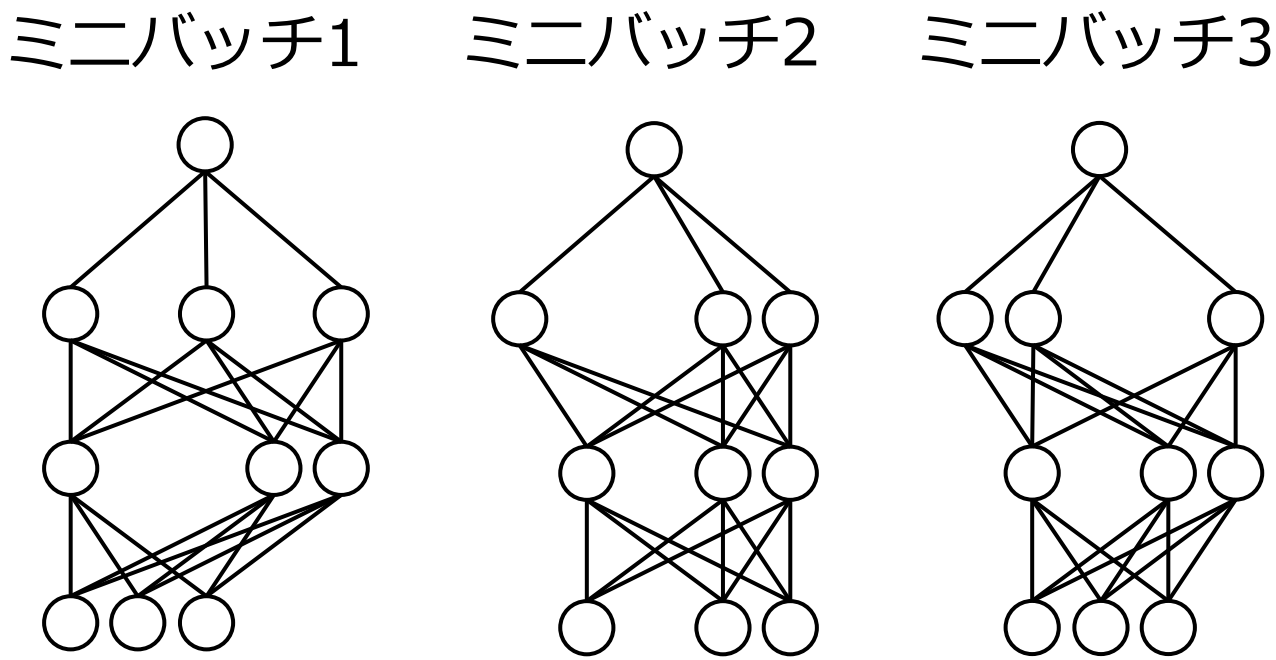
©Yutaka Matsuo. All rights reserved.

ドロップアウト/ドロップコネクトの利点

- 利点１：巨大NNの過学習を抑制できる
- 利点２：擬似的にアンサンブル学習をしているとみなせる



Wan, Li, et al., “Regularization of neural networks using dropconnect”, International conference on machine learning, PMLR, 2013. より図を引用



©Yutaka Matsuo. All rights reserved.

目次

- 最適化アルゴリズム（Optimizer）
- パラメータの初期化戦略
- 正規化
- 正則化
- **実践的な方法論**
- 深層学習ライブラリ・CPU・GPU

©Yutaka Matsuo. All rights reserved.

教師なし学習

教師なし学習とは，教師ラベルを必要としない学習手法のこと
データの特徴や構造を学習するのに適している

- **クラスタリング（Clustering）**
  - クラス分類と似ているが，教師ラベルにより分類するのではなく，データ間の距離などを用いて似ているデータを同じクラスとなるように分類する
  - K-means法，DBSCANなど

- 次元削減
  - 入力データの特徴をできるだけ保持しつつ次元を減らす
  - 主成分分析，オートエンコーダなど

©Yutaka Matsuo. All rights reserved.

半教師あり学習

- **半教師あり学習（Semi-Supervised Learning）**の枠組みでは，$p(\boldsymbol{x})$ のラベルなしデータ（集めやすい）と $p(\boldsymbol{x}, \boldsymbol{y})$ のラベルありデータ（集めにくい）の両方が $p(\boldsymbol{y}|\boldsymbol{x})$ の推定や $\boldsymbol{x}$ から $\boldsymbol{y}$ を予測するのに使用される

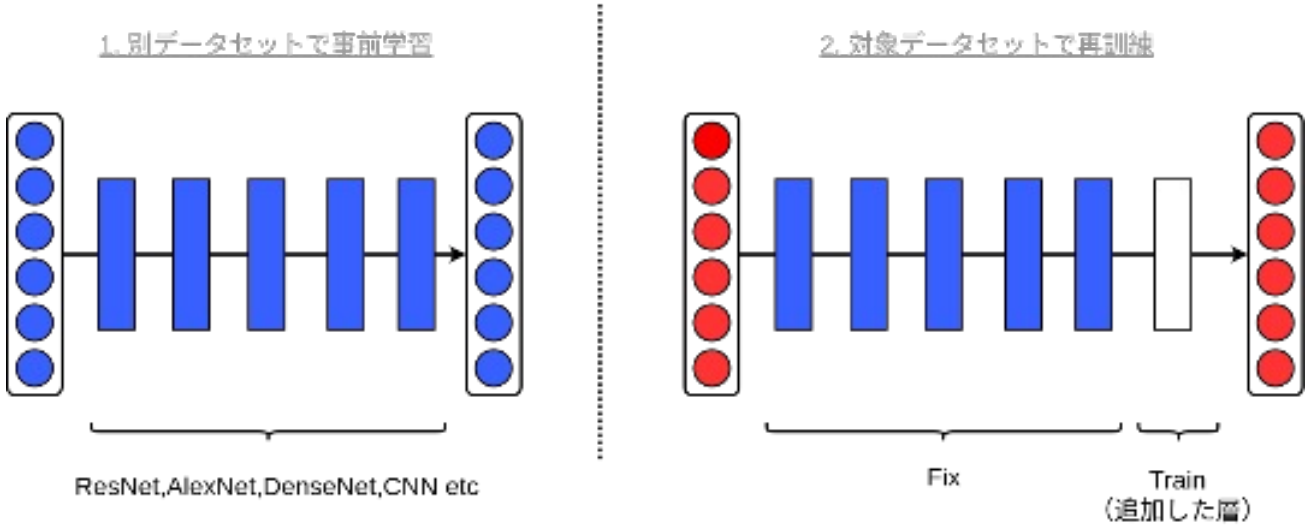
- 深層学習の観点では半教師あり学習は通常，表現 $\boldsymbol{h} = f(\boldsymbol{x})$ を学習することを指す．この際目標となるのは，同じクラスのデータが似たような表現 $\boldsymbol{h}$ を持つように学習させること
  すなわち入力空間で近くに集まっているデータは類似した表現に写像されるべきである

©Yutaka Matsuo. All rights reserved.

転移学習

- **転移学習（Transfer Learning）**は「新規タスクの効果的な仮説を効率的に見つけ出すために，一つ以上の別のタスクで学習された知識を得て，それを適用する問題」
（Inductive Transfer：10 Years Later．NIPS 2005 Workshopより）

- **ファインチューニング（Fine-Tuning）**



  - 深層学習では，別データセットで学習したモデルのパラメータを固定し，追加した層のみを対象の問題のデータセットで学習する
  - 一部のパラメータのみ学習することは特にファインチューニングとも呼ばれる
  - 対象の問題を学習するためのデータセットを減らすことができ，汎化性能を大きく下げることなく学習時間を短縮できる

©Yutaka Matsuo. All rights reserved.

転移学習

- ファインチューニングは，データ分布が離れているとうまくいかない
  - 例：Distributional Shift [Shankar+ 2018]
    - MNIST->回転MNIST



(a) M (b) $M_{15°}$ (c) $M_{30°}$



(d) $M_{45°}$ (e) $M_{60°}$ (f) $M_{75°}$

- 学習データとテストデータの分布が一致しない状況は**ドメインシフト（Domain Shift）**と呼ばれ，それに対処する手法が盛んに研究されている
- 具体例
  - Domain Adversarial Net（DAN） [Ganin+ 2016]
  - CrossGrad [Shankar+ 2018]

- https://www.slideshare.net/YuusukeIwasawa/dl-asymmetric-tritraining-for-unsupervised-domain-adaptation-icml2017-and-neural-domain-adaptation

Key Words
- Transfer Learning
- Domain Adaptation
- Domain Generalization

©Yutaka Matsuo. All rights reserved.

ニューラルネットの構成要素まとめ

- ネットワーク構造
  - 全結合層，畳み込み（第5回で説明），RNN（第7回で説明），
    Attention（第7回で説明），Residual構造，活性化関数の種類，畳み込みのチャネル数…
- 目的関数（タスクによってある程度しぼられる）
  - 交差エントロピー，MSE，Triplet損失（補足を参照）…
- 最適化手法（optimizerなど）
  - SGD，モメンタム，Adam，バッチサイズ，
    学習率，エポック数，早期終了…
- テクニック
  - バッチ正規化，Dropout，データ拡張（第6回で説明），入力正規化
    補助タスクの設定，L2正則化，アンサンブル

©Yutaka Matsuo. All rights reserved.

ハイパーパラメータ

- 一般的なもの
  - 学習率やバッチサイズ，エポック数
- ネットワーク構造やタスクに応じて：
  - 重み減衰の係数（正則化項の係数）α など
  - 隠れ層のニューロンの数
  - 隠れ層の数（レイヤー総数）

- [実践] 実験管理
  - ハイパーパラメータを.yamlファイルなどで一括管理する
    （Hydraなど扱いやすいツール多数）
  - TensorboardやW&Bなどを使い実験を可視化・管理する

表 11.1: さまざまなハイパーパラメータがモデルの能力に与える影響

ハイパーパラメータ	能力が向上する場合...	理由	注意
隠れユニットの数	増加させたとき	隠れユニットの数を増やすと，モデルの表現能力が向上する．	隠れユニットの数を増やすと，本質的にモデルに関連するすべての処理で実行時間とメモリコストが増加する．
学習率	最適にチューニングされたとき	学習率が不適切な場合，それが高すぎても低すぎても，最適化の失敗して有効容量が低いモデルになる．	
畳み込みカーネルの幅	増加させたとき	カーネル幅を大きくすると，モデルの中のパラメータ数が増える．	カーネル幅が広いほど，出力の次元が減り，暗黙的なゼロパディングを使用してこの影響を減らさない限り，モデルの能力は劣化する．カーネル幅が広くなると，パラメータの格納に必要なメモリ量と実行時間が増加するが，出力の次元が減ればメモリコストは削減される．
暗黙的なゼロパディング	増加させたとき	畳み込みの前に暗黙的なゼロを追加して，表現サイズを大きいまま保持する．	ほとんどの処理の実行時間とメモリコストが増加する．
重み減衰の係数	減少させたとき	重み減衰の係数を小さくすると，モデルパラメータを自由に大きくできる．	
ドロップアウトの割合	減少させたとき	ユニットをドロップする頻度が減ると，ユニットは訓練集合に適合するためにお互いに「協調する」機会が増える．	

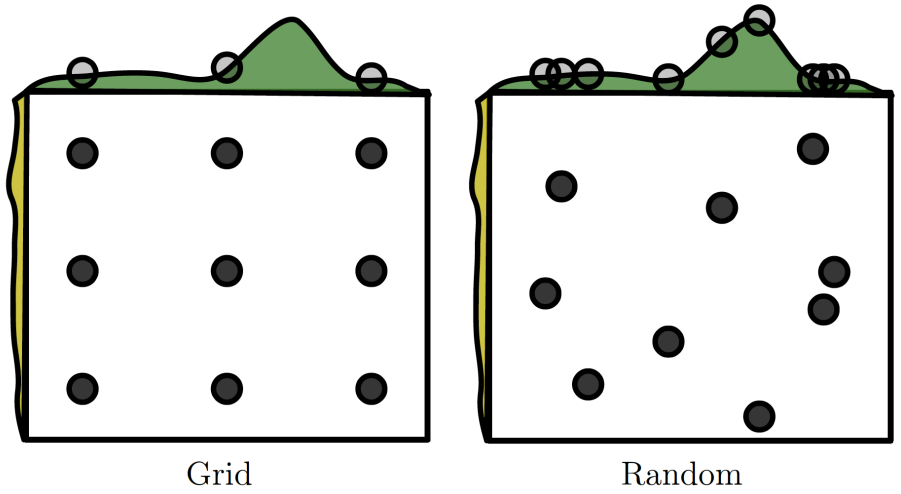
©Yutaka Matsuo. All rights reserved.

グリッドサーチとランダムサーチ [Bergstra & Bengio 2012]

- ハイパーパラメータの組み合わせによってモデルの性能がかなり変わるため，最適な組み合わせをいかに見つけるかが重要

- **グリッドサーチ（Grid Search）**
  - あらかじめ指定したハイパーパラメータの組み合わせをすべて試す
  - 長所：網羅的に探索するので最適な組み合わせが確実に見つかる
  - 短所：探索に時間がかかる

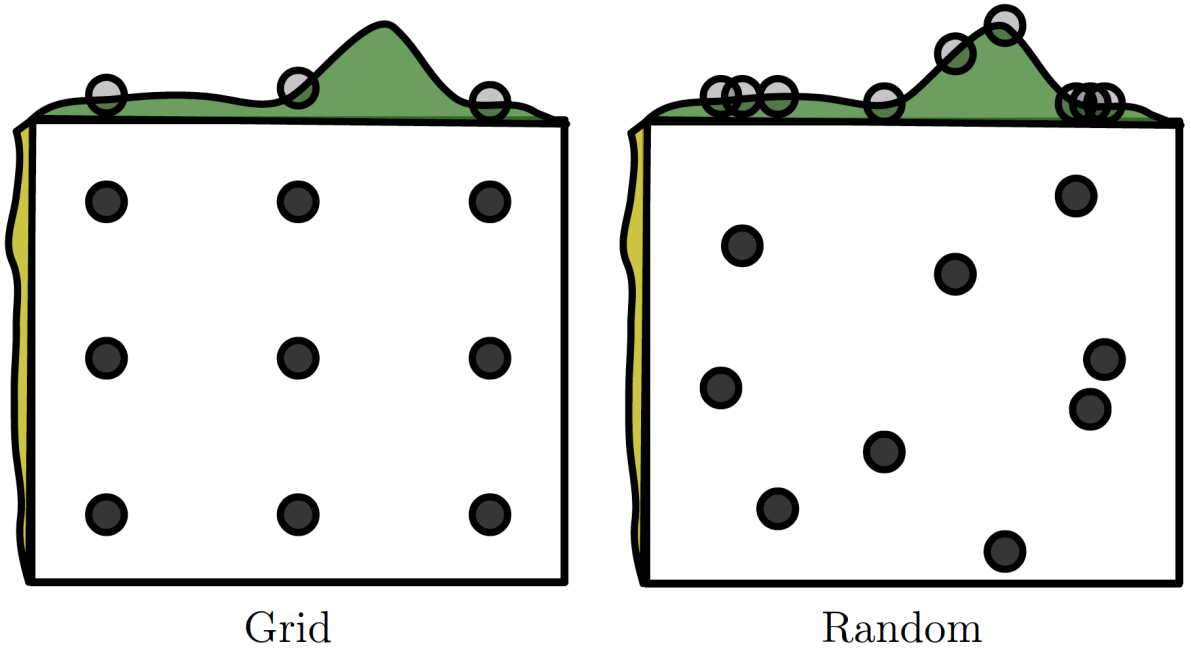
- **ランダムサーチ（Random Search）**
  - 指定した回数だけ組み合わせの中からランダムに探索
  - 長所：探索時間を削減できる
  - 短所：最適な組み合わせを見つけられる確率が減る



Bergstra & Bengio, “Random Search for Hyper-Parameter Optimization” Journal of Machine Learning Research (2012)

©Yutaka Matsuo. All rights reserved.

グリッドサーチとランダムサーチ



Bergstra & Bengio, “Random Search for Hyper-Parameter Optimization” Journal of Machine Learning Research (2012)

- 解空間が広い&特定のハイパラのみ重要な場合，ランダムサーチのほうが良い解にたどり着く
- ただし，探索するハイパラが膨大になるとランダムサーチでも非現実的
  - ハイパラのセンシティビティだけを調べた研究もある
  - [Greff+ 2016] “LSTM: A Search Space Odyssey”
- パラメータ探索時には検証データを使う（テストデータを使わない）こと

©Yutaka Matsuo. All rights reserved.

Neural Architecture Search

- **Neural Architecture Search（NAS）** [Zoph & Le 2016]
  - ネットワークアーキテクチャを半自動で決める
  - AutoMLなどとも呼ばれる

  - パラメータを出力するネットワークを用意し，そのネットワークも学習する
    - 最終的な精度が改善するようなパラメータを出力するように最適化
    - 強化学習を使うことが多い（例外もある）

  - 例：NASNet [Zoph+ 2018], ENAS [Pham+ 2018], BayesNAS [Zhou+ 2019], DARTS [Chen+ 2021] など

©Yutaka Matsuo. All rights reserved.

可視化して確認することも重要

- 定性的な確認
  - ネットワークを可視化する
  - 出力結果を見る（生成画像なら生成したもの，予測なら予測）

- 定量的な確認
  - 学習曲線を見る

©Yutaka Matsuo. All rights reserved.

目次

- 最適化アルゴリズム（Optimizer）
- パラメータの初期化戦略
- 正規化
- 正則化
- 実践的な方法論
- **深層学習ライブラリ・CPU・GPU**

©Yutaka Matsuo. All rights reserved.

深層学習ライブラリ

- **PyTorch**
  - 開発：Facebook AI Research（FAIR）
  - Pythonのエコシステムによく適合し，Python-likeにコードを書くことができる
  - 低レベルAPIも充実し柔軟な実装が可能
  - **動的計算グラフ（Define-by-Run）**
    - 計算グラフの構築とデータの処理が同時に行われる方式．途中の処理結果に応じて，処理を変更できる
  - その他
    - PyTorch Lightningによる実装作業の簡略化
    - Hugging Faceによる実装済み/訓練済みモデル

※本講義ではPyTorchを利用します

©Yutaka Matsuo. All rights reserved.

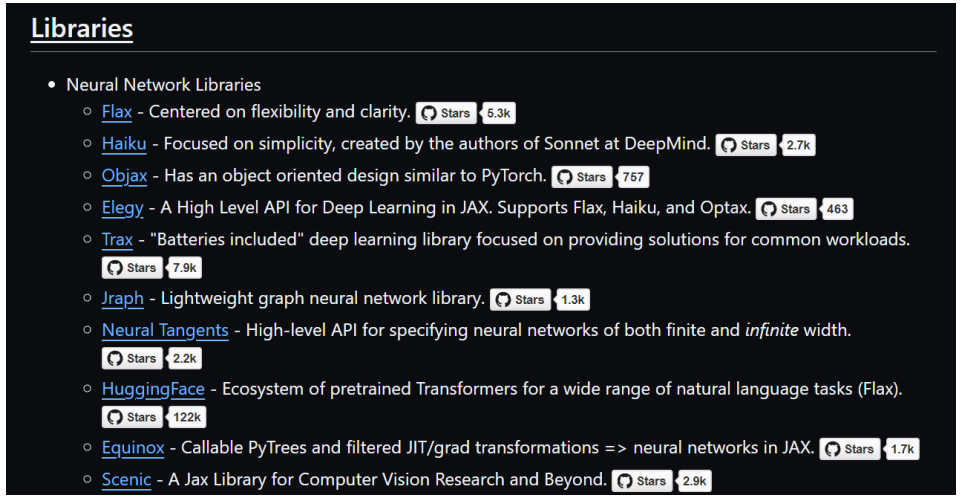
深層学習ライブラリ

- **TensorFlow**
  - 開発：Google
  - Kerasによる高レベルAPIにより，初心者に優しい
    - ただし柔軟な実装が難しい
  - **静的計算グラフ（Define-and-Run）**
    - 計算グラフを構築した後にデータを流し処理を実行する方式．最適化しやすいが，途中経過の処理を確認しづらい
    - ただしバージョン2.0以降，Eager Executionによる動的計算グラフ（define-by-run)がデフォルトに
  - その他
    - XLA（Accelerated Linear Algebra）によるGPU/TPUでの計算高速化
    - TensorFlow.jsやTensorFlow Liteによるウェブアプリケーション化やエッジ化の容易さ
    - Kerasはバージョン3以降，バックエンドとしてPyTorchやJAX（後述）も選択できるようになった

©Yutaka Matsuo. All rights reserved.

深層学習ライブラリ

- JAX
  - 開発：Google
  - JAX自体は低レベルAPIで，JAXを使ったFlax（Google）やHaiku（DeepMind）などが，PyTorchやTensorFlowに対応するものとみなせる
  - 主な機能
    - @jit (just in time): コードをコンパイルして実行し高速化
    - grad(): 自動微分
    - vmap(): 関数をベクトル化し自動で並列処理に対応させる



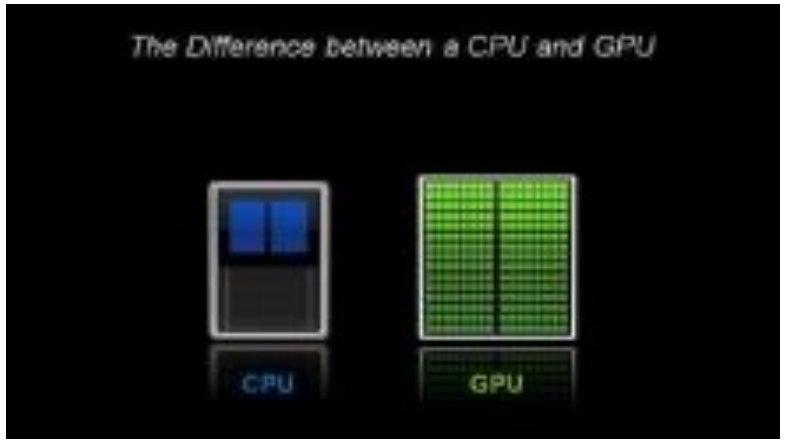
https://github.com/n2cholas/awesome-jax

©Yutaka Matsuo. All rights reserved.

デバイス（GPU）による高速化

- **GPU（Graphics Processing Unit）**はディスプレイへの描画処理を高速に行う装置として発達してきたが，近年ではグラフィクス以外の用途への活用も広まっている
  - **GPGPU（General-Purpose Computing on Graphics Processing Units）**は科学技術計算などのグラフィクス処理以外の汎用的な計算にGPUを応用する技術を指す
- 一般にCPU（Central Processing Unit）は直列計算，GPUは並列計算を得意とし，メモリに関してはCPUではメモリ容量が重要視される傾向にある一方，GPUでは単位時間あたりのデータ読み出し量が重視される
  - CPUとGPUではコア当たりの計算性能はCPUに軍配が上がるが，多数のコアを搭載することで特定の計算においてはGPUがCPUを超える性能を出すことができる

数個のコア



~10000個のコア

https://blogs.nvidia.com/blog/2009/12/16/whats-the-difference-between-a-cpu-and-a-gpu/

©Yutaka Matsuo. All rights reserved.

CPU・GPUの比較

CPU	GPU
高性能なコアが少数 直列処理が得意 複雑な計算に適する	低性能なコアが多数 並列処理が得意 単純計算に適する

深層学習では単純な行列演算を膨大な回数行うため，GPUの利点が生きる

©Yutaka Matsuo. All rights reserved.

TPU

- **TPU（Tensor Processing Unit）**
  - Googleが開発した，特定用途向けの集積回路

- ディープラーニングの学習や推論に最適化されている
  - 数千個の乗算アキュムレータを搭載
    - アキュムレータとは，演算の際に被演算子の値を保持するメモリ
  - 演算結果を回路内で直接受け渡し可能
    - ※GPUはメモリにアクセスして中間結果を保存しなければならない
  - → メモリアクセスが大幅に減るため，大規模な行列演算を極めて高速に実行できる

- TensorFlowではTPUの使用をサポート（PyTorchでも可）
- 2025年9月現在，第7世代（Ironwood）まで登場している
- Google ColabやKaggle Notebookなどでは使用可能

©Yutaka Matsuo. All rights reserved.

まとめ

- 最適化アルゴリズム（Optimizer）
  - SGD, モメンタム, ネステロフの加速法, Adam, …
- パラメータの初期化
  - Xavierの初期化, Heの初期化
- 正規化
  - バッチ正規化，Layer正規化，Instance正規化，Group正規化
- 正則化
  - パラメータノルムによる正則化（L1, L2），スパース表現，早期終了，アンサンブル，ドロップアウト，ドロップコネクト
- 実践的な方法論
  - 教師なし学習，半教師あり学習，転移学習，ハイパーパラメータ，グリッドサーチとランダムサーチ，NAS
- 深層学習ライブラリ・CPU・GPU

©Yutaka Matsuo. All rights reserved.

今回の目的と目標

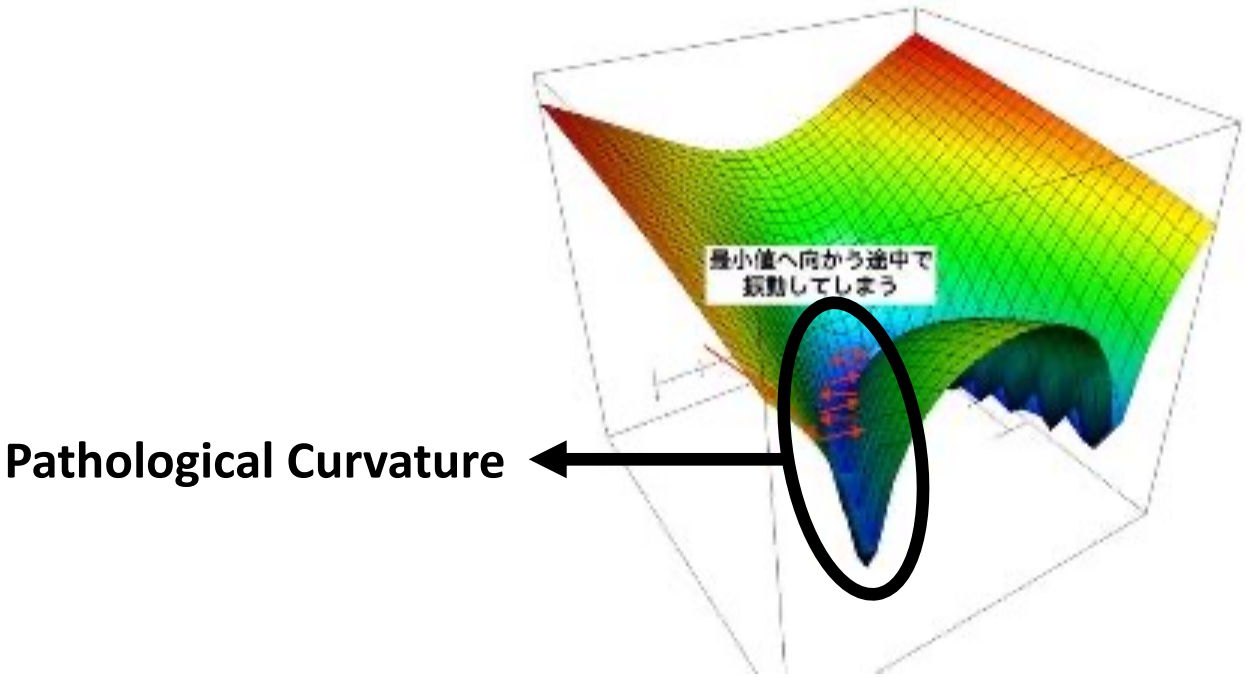
- **目的**
  - ニューラルネットワークの基礎を踏まえ，最適化・初期化・正則化などのより実践的な内容を理解する

- **目標**
  - 確率的勾配降下法（SGD）や各種最適化手法について基本的な概念を説明できる
  - メジャーなパラメータ初期化方法について説明できる
  - メジャーな正規化手法について説明できる
  - 様々な過学習を抑える手法について説明できる

©Yutaka Matsuo. All rights reserved.

補足：Pathological Curvature

- **Pathological Curvature**
  - 機械学習によって最適化を行う際に，最適化対象となるコスト関数の表面におけるくぼみが鋭く，最適化が困難になる（解探索経路が振動する）箇所がある
    この箇所をPathological Curvatureという
  - ミニバッチ学習SGD法などによって学習データ数を抑えると，外れ値（ノイズ）の影響を受けやすいため，Pathological Curvatureにハマりやすいと言われている



©Yutaka Matsuo. All rights reserved.

補足：学習率の調整

- **陰的正則化（Implicit Regularization）**
  - L1正則化，L2正則化のように明示的な正則化ではなく，ニューラルネットワークの構造自体が持つ自動的な正則化機能のこと

- 学習率
  - 機械学習の最適化における値の変動の大きさを表す値
  - 大きすぎる
    - ○ 最適化の速度は早い
    - × 値が発散したり収束値の精度が悪化する
  - 小さすぎる
    - ○ 収束値，最適解の精度が良い傾向にある
    - × 最適化速度が遅い．過学習しやすい

Adamなどの最適化アルゴリズムや
Optunaなどのハイパーパラメータの
最適化ライブラリを用いることで，
最適化アルゴリズム（RMSProp, Adagradなど）における
各パラメータに対して個別の学習率を計算し，
適切な学習率のチューニングが可能となる

©Yutaka Matsuo. All rights reserved.

補足：ノイズに対する頑健性（Robustness）

- ラベル y が必ずしも正しいとは限らない．その場合，ラベルに生じうるノイズをモデル化することが有効なことがある
- 例えば，ラベル y が確率 $1-\varepsilon$ で正しく，確率 ε で間違っているという仮定を，解析的にコスト関数に組み入れることを考える：**ラベル平滑化（Label Smoothing）**
  - ラベル平滑化では，クラスのラベルであるOne-hotベクトルの要素0と1を，それぞれ $\varepsilon/(k-1)$ と $1-\varepsilon$ に置き換えることで，k 個の出力値を持つソフトマックスに基づいたモデルの正則化を行う
  - 交差エントロピーをこれらの**Soft Target**（例：[0.7, 0.1, 0.1]）に対して使用できる
  - ソフトマックスと**Hard Target**（例：[1, 0, 0]）による最尤推定は収束しない場合がある．これは，ソフトマックスが0または1ちょうどの値を出力できず，学習する重みが大きくなり続けてしまうためである
    →（重み減衰など他の正則化手法で防ぐことも可能だが）
    Soft targetの利用により改善が期待できる

©Yutaka Matsuo. All rights reserved.

補足：マルチタスク学習

- **マルチタスク学習（Multi-task Learning）**は，モデルが複数のタスクに同時に取り組むことで，汎化性能の向上を目指す方法
  - パラメータに課せられた緩やかな制約とみなせる

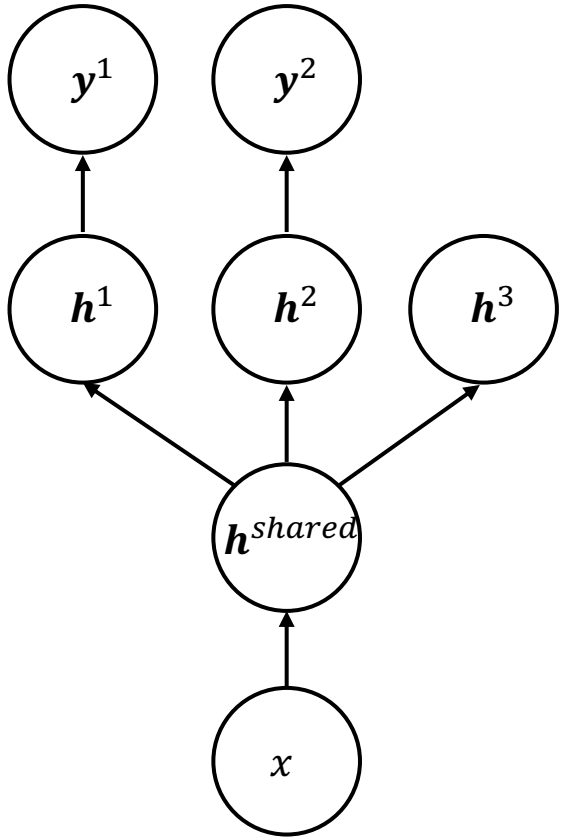
- モデルの一部がタスク間で共有されると，共通部分が複数タスク全体で改善されるように学習するため，汎化性能が向上することが多い
  - 訓練事例を追加すると汎化性能が向上するのと同様

©Yutaka Matsuo. All rights reserved.

補足：マルチタスク学習

- 下図の例ではタスク固有のパラメータ($\boldsymbol{h}^1, \boldsymbol{h}^2$)とタスク間で共有される
  パラメータ $\boldsymbol{h}^{shared}$ が存在している

- 深層学習の観点から見ると，その根底にある
  事前信念は，異なるタスクに関連するデータで
  観測される変動を説明する因子の中には，
  2つ以上のタスクの間で共有されるものが
  いくつか存在するということである



Deep Learning (Goodfellow, Bengio & Courville)

©Yutaka Matsuo. All rights reserved.

参考文献

[Nesterov 1983] “A method for solving the convex programming problem with convergence rate O (1/k2)” Dokl Akad Nauk SSSR 269 543-, 1983

[Duchi+ 2011] “Adaptive Subgradient Methods for Online Learning and Stochastic Optimization” The Journal of Machine Learning Research, Volume 12 :2121 – 2159

[Tieleman & Hinton 2012] “Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude” COURSERA: Neural Networks for Machine Learning 4 2], 26-, 2012

[Kingma & Ba 2014] “Adam: A Method for Stochastic Optimization” arXiv:1412.6980

[Dozat 2016] “Incorporating Nesterov Momentum into Adam" International Conference on Learning Representations 2016 Workshop Track

[Reddi+ 2019] ”On the Convergence of Adam and Beyond” arXiv: 1904.09237

[Bengio+ 2007] “Greedy Layer-Wise Training of Deep Networks” In Advances in neural information processing systems pp. 153-160].

[Glorot & Bengio 2010] “Understanding the difficulty of training deep feedforward neural networks” Proceedings of Machine Learning Research, 9, 249-256

[He+ 2015] “Delving deep into rectifiers: Surpassing human-level performance on imagenet classification” arXiv:1502.01852

[Ioffe & Szegedy 2015] “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift” arXiv:1502.03167

[Wu & He 2018] “Group Normalization” arXiv:1803.08494

[Ba+ 2016] “Layer Normalization” arXiv:1607.06450

[Ulyanov+ 2016] “Instance Normalization: The Missing Ingredient for Fast Stylization” arXiv:1607.08022

[Srivastava+ 2014] “Dropout: A Simple Way to Prevent Neural Networks from Overfitting” Journal of Machine Learning Research 15 2014] 1929-1958

[Wan+ 2013] “Regularization of Neural Networks using DropConnect” Proceedings of the 30th International Conference on Machine Learning, PMLR 28 3]:1058-1066, 2013.

[Ganin+ 2016] “Domain-Adversarial Training of Neural Networks” arXiv:1505.07818

[Shankar+ 2018] “Generalizing Across Domains via Cross-Gradient Training” arXiv:1804.10745

[Bergstra & Bengio 2012] “Random Search for Hyper-Parameter Optimization” Journal of Machine Learning Research 13 281-305

[Greff+ 2016] “LSTM: A Search Space Odyssey” IEEE transactions on neural networks and learning systems 28.10 (2016): 2222-2232.

[Zoph & Le 2016] “Neural Architecture Search with Reinforcement Learning” arXiv:1611.01578

[Zoph+ 2018] “Learning Transferable Architectures for Scalable Image Recognition” Proceedings of the IEEE conference on computer vision and pattern recognition. 2018.

[Pham+ 2018] “Efficient Neural Architecture Search via Parameter Sharing” arXiv:1802.03268

[Zhou+ 2019] “BayesNAS: A Bayesian Approach for Neural Architecture Search” International conference on machine learning. PMLR, 2019.

[Chen+ 2021] “Progressive DARTS: Bridging the Optimization Gap for NAS in the Wild” International Journal of Computer Vision129 (2021): 638-655.

©Yutaka Matsuo. All rights reserved.

©Yutaka Matsuo. All rights reserved.